

UNIT 2

Data Modelling and Relational Database Design

2. Data Modelling and Relational Database Design

2.1 Data Modelling using ER Diagram: Representation of Entities, Attributes, Relationships and their Type, Cardinality, Generalization, Specialization, Aggregation.

2.2 Relational data model: Structure of Relational Database Model, Types of keys, Referential Integrity Constraints

2.3 Codd's rules

2.4 Database Design using E-R, E-R to Relational

2.5 Normalization – Normal forms based on primary
(1 NF, 2 NF, 3NF, BCNF)

- Data modeling is **the process of creating a simplified diagram of a software system and the data elements it contains, using text and symbols to represent the data and how it flows.**
- Data models provide a blueprint for designing a new database or reengineering a legacy application.
- Data modeling helps an organization use its data effectively to meet business needs for information.

Data modeling (cont...)

- A data model can be thought of as a flowchart that illustrates data entities, their attributes and the relationships between entities.
- It enables data management and analytics teams to document data requirements for applications and identify errors in development plans before any code is written.

Data modeling (cont...)

- A data model can be thought of as a flowchart that illustrates data entities, their attributes and the relationships between entities.
- It enables data management and analytics teams to document data requirements for applications and identify errors in development plans before any code is written.

- The **Data Model** is defined as an abstract model that organizes data description, data semantics, and consistency constraints of data.
- Data Model is like an architect's building plan, which helps to build conceptual models and set a relationship between data items.
- The two types of Data Modeling Techniques are
 - Entity Relationship (E-R) Model
 - UML (Unified Modelling Language)

Why use Data Model?

- Ensures that all data objects required by the database are accurately represented.
- A data model helps design the database at the conceptual, physical and logical levels
- Data Model structure helps to define the relational tables, primary and foreign keys and stored procedures.
- It provides a clear picture of the base data and can be used by database developers to create a physical database.
- It is also helpful to identify missing and redundant data.

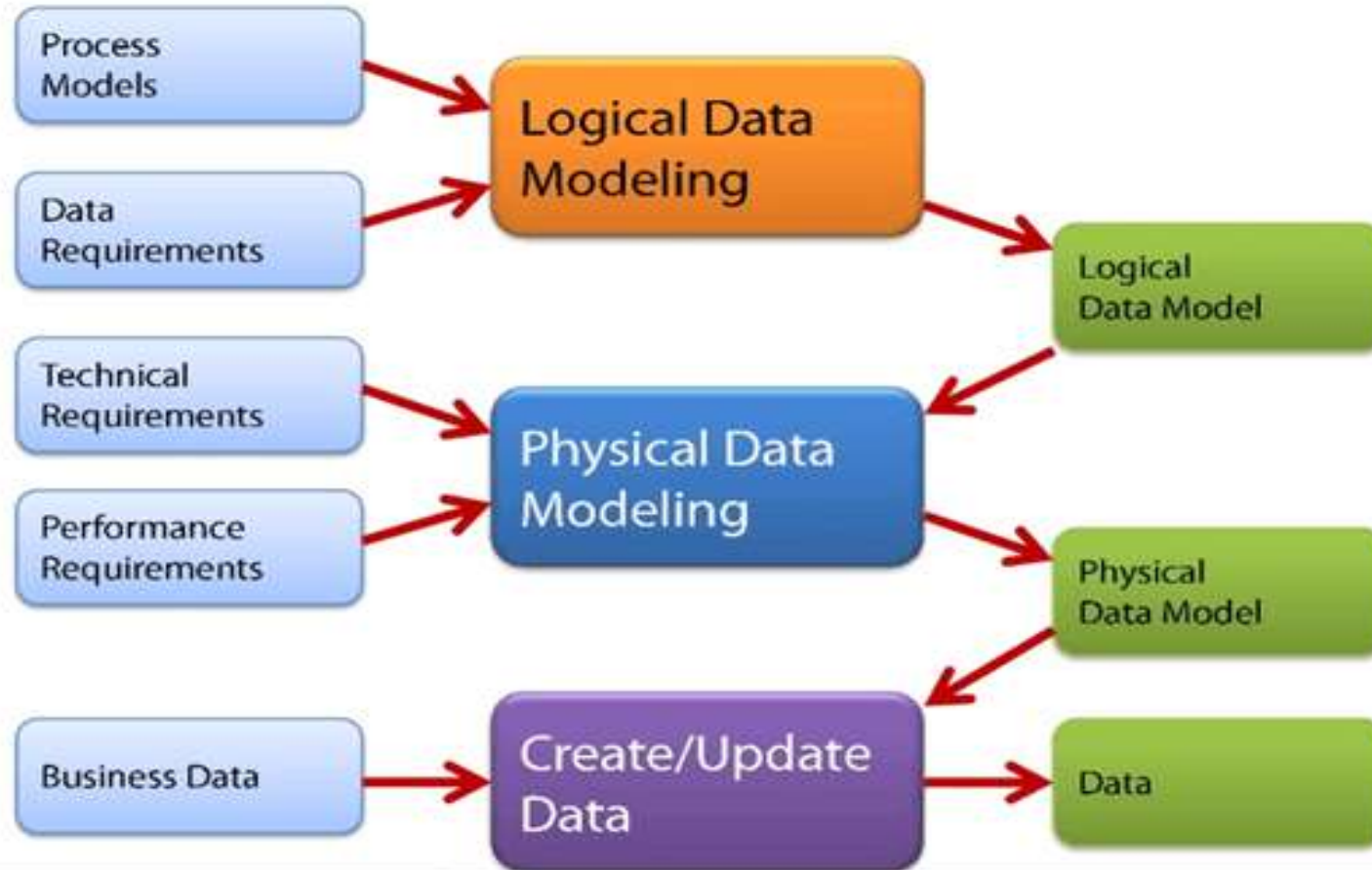
Types of Data Models

- Mainly three different types of data models:
- **conceptual data models**, **logical data models**, and **physical data models**,.
- The data models are used to represent the data and how it is stored in the database and to set the relationship between data items.

Types of Data Models (cont..)

- **Conceptual Data Model:** This Data Model defines **WHAT** the system contains. This model is typically created by Business stakeholders and Data Architects. The purpose is to organize, scope and define business concepts and rules.
- **Logical Data Model:** Defines **HOW** the system should be implemented regardless of the DBMS. This model is typically created by Data Architects and Business Analysts. The purpose is to developed technical map of rules and data structures.
- **Physical Data Model:** This Data Model describes **HOW** the system will be implemented using a specific DBMS system. This model is typically created by DBA and developers. The purpose is actual implementation of the database.

Types of Data Models (cont..)



ENTITIES, ATTRIBUTES, AND ENTITY SETS

- **Entity** : An entity is a thing in a real-world with independent existence. An entity can exist independently and is distinguishable from other objects. It can be identified uniquely.
- An entity can be of two types :
 - **Tangible Entity** : Entities that exist in the real world physically.
Example: Person, car, etc.
 - **Intangible Entity** : Entities that exist only logically and have no physical existence. Example: Bank Account, etc.

ENTITIES, ATTRIBUTES, AND ENTITY SETS

- **ENTITIES**

- Entity in DBMS can be a real-world object with an existence, For example, in a **College** database, the entities can be Professor, Students, Courses, etc.
- Entities has attributes, which can be considered as properties describing it, for example, for Professor entity, the attributes are **Professor_Name**, **Professor_Address**, **Professor_Salary**, etc. The attribute value gets stored in the database.

ENTITIES, ATTRIBUTES, AND ENTITY SETS

- **ENTITIES**

- Entity in DBMS can be a real-world object with an existence, For example, in a **College** database, the entities can be Professor, Students, Courses, etc.
- Entities has attributes, which can be considered as properties describing it, for example, for Professor entity, the attributes are **Professor_Name**, **Professor_Address**, **Professor_Salary**, etc. The attribute value gets stored in the database.

ENTITIES, ATTRIBUTES, AND ENTITY SETS

Example of Entity in DBMS

Professor_ID	Professor_Name	Professor_City	Professor_Salary
P01	Tom	Sydney	\$7000
P02	David	Brisbane	\$4500
P03	Mark	Perth	\$5000

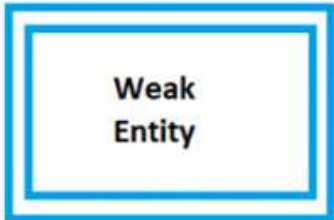
<Professor>

Here, Professor_Name, Professor _Address and Professor _Salary are attributes. Professor_ID is the primary key

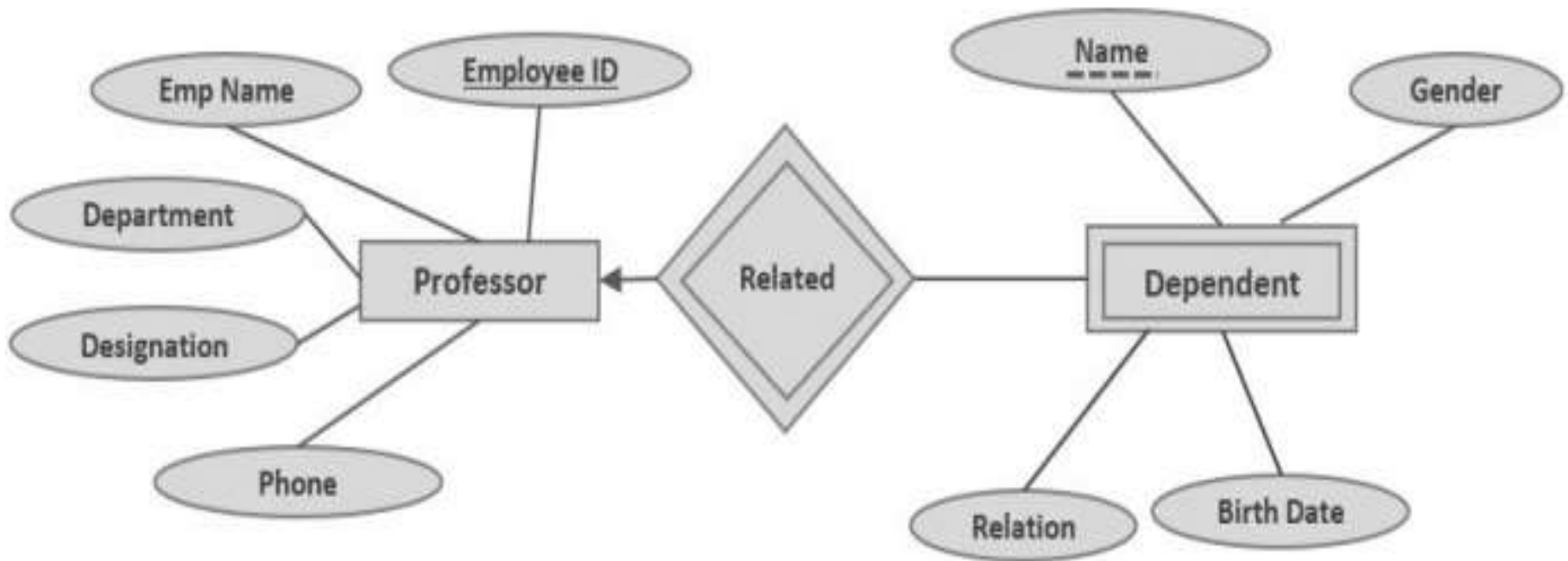
Types of DBMS Entities

- **Strong Entity**
- The strong entity has a primary key. Weak entities are dependent on strong entity. Its existence is not dependent on any other entity.
- Strong Entity is represented by a single rectangle – A diagram showing a blue rectangular box with the text "Strong Entity" inside, representing the symbol for a strong entity in database notation.
- **Professor** is a strong entity here, and the primary key is **Professor_ID**.

Weak Entity

- The weak entity in DBMS do not have a primary key and are dependent on the parent entity. It mainly depends on other entities.
- Weak Entity is represented by double rectangle – A diagram showing a rectangle with a double blue border, representing a weak entity. The text "Weak Entity" is centered inside the rectangle.
- **Professor** is a strong entity, and the primary key is **Professor_ID**. However, another entity is **Professor_Dependents**, which is our Weak Entity.
- This is a weak entity since its existence is dependent on another entity **Professor**, A Professor has Dependents.

Example of Strong and Weak Entity

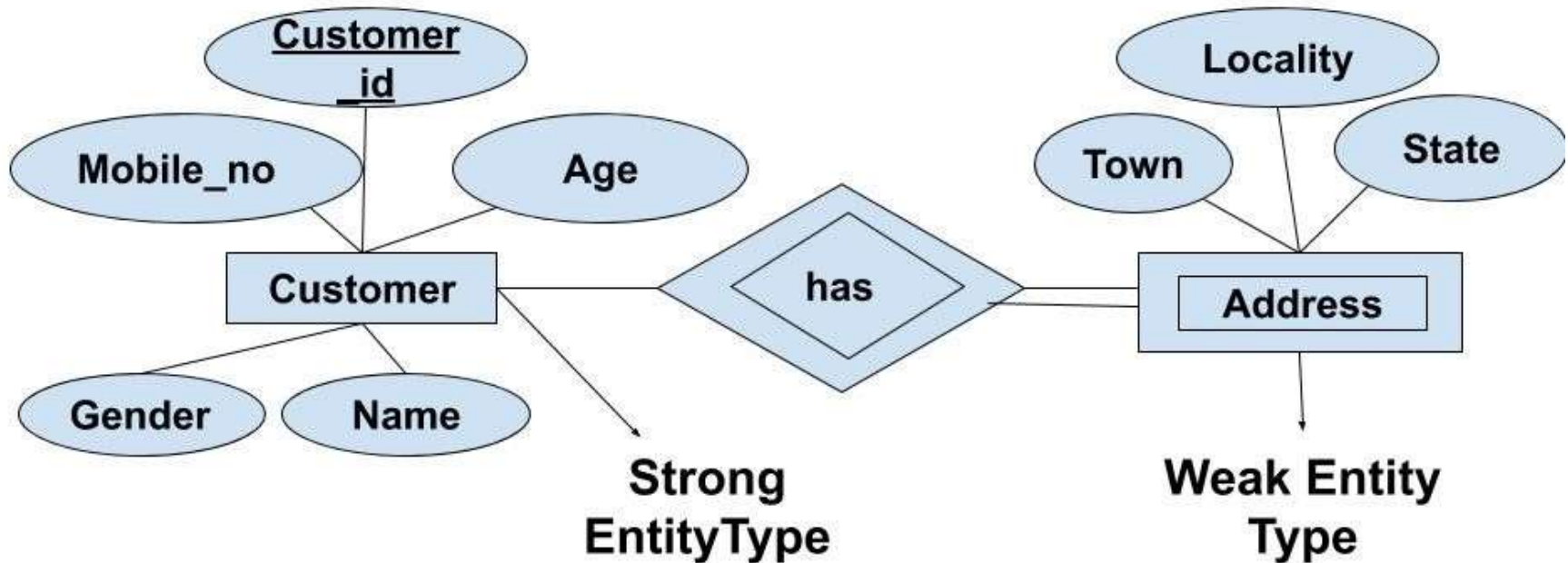


The Strong Entity is **Professor**, whereas **Dependent** is a Weak Entity. **ID** is the primary key (represented with a line) and the Name in **Dependent** entity is called **Partial Key** (represented with a dotted line).

Solve Example of Strong and Weak Entity

- **Example:** *If we have two tables of Customer(Customer_id, Name, Mobile_no, Age, Gender) and Address(Locality, Town, State, Customer_id). Here we cannot identify the address uniquely as there can be many customers from the same locality. So, for this, we need an attribute of Strong Entity Type i.e 'Customer' here to uniquely identify entities of 'Address' Entity Type.*

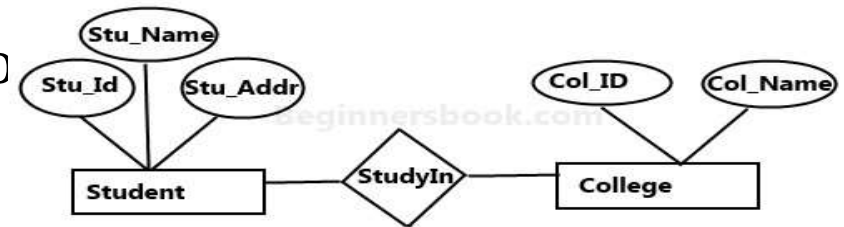
Example of Strong and Weak Entity



The Strong Entity is **Professor**, whereas **Dependent** is a Weak Entity. **ID** is the primary key (represented with a line) and the Name in **Dependent** entity is called **Partial Key** (represented with a dotted line).

Example of ER Diagram

- **Example:** Student entity has attributes such as Stu_Id, Stu_Name & Stu_Addr and College entity has attributes such as Col_ID & Col_Name.



Sample E-R Diagram

Rectangle: Represents Entity sets.

Ellipses: Attributes

Diamonds: Relationship Set

Lines: They link attributes to Entity Sets and Entity sets to Relationship Set

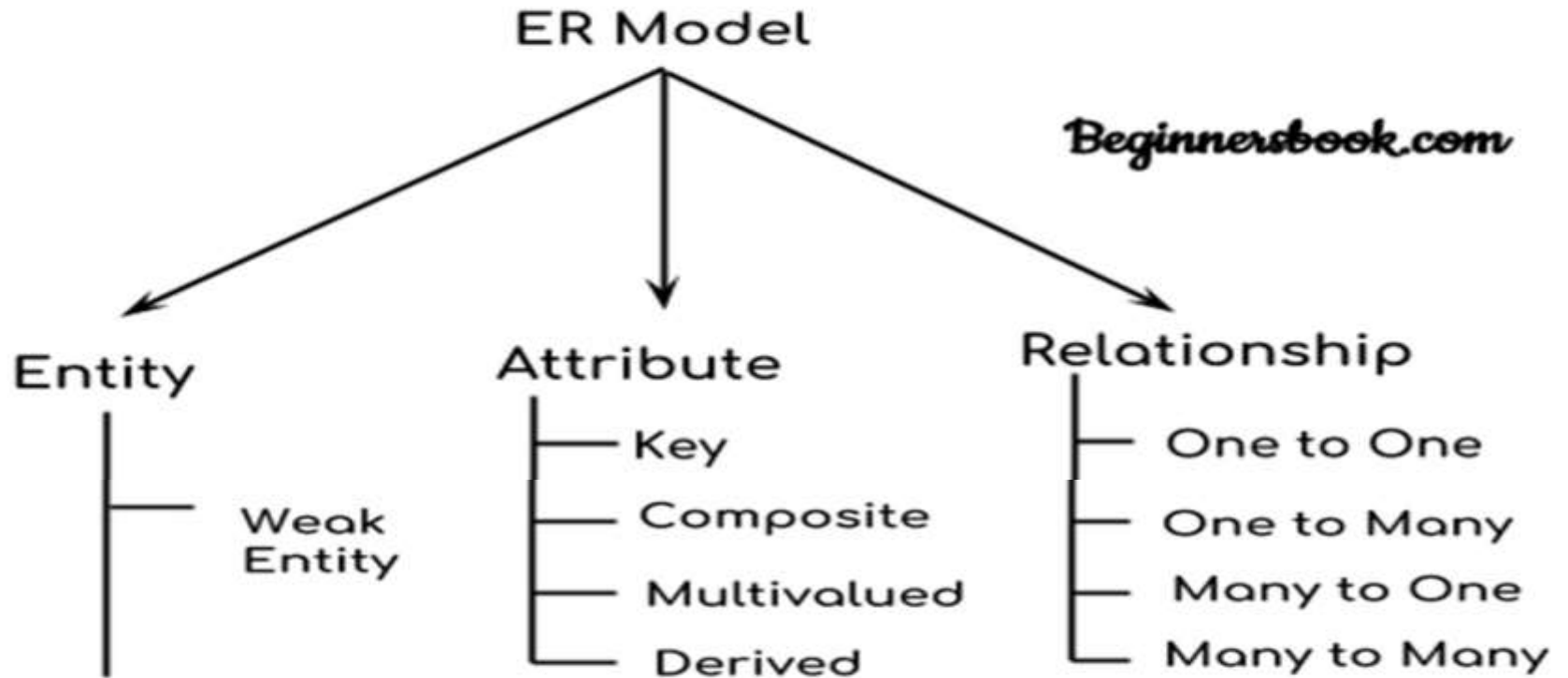
Double Ellipses: Multivalued Attributes

Dashed Ellipses: Derived Attributes

Double Rectangles: Weak Entity Sets

Double Lines: Total participation of an entity in a relationship set

Components of a ER Diagram



Components of ER Diagram

ER diagram has three main components:

1. Entity

2. Attribute

3. Relationship

1. Entity

An entity is an object or component of data. An entity is represented as rectangle in an ER diagram. For example: In the following ER diagram we have two entities Student and College and these two entities have many to one relationship as many students study in a single college. We will read more about relationships later, for now focus on entities.

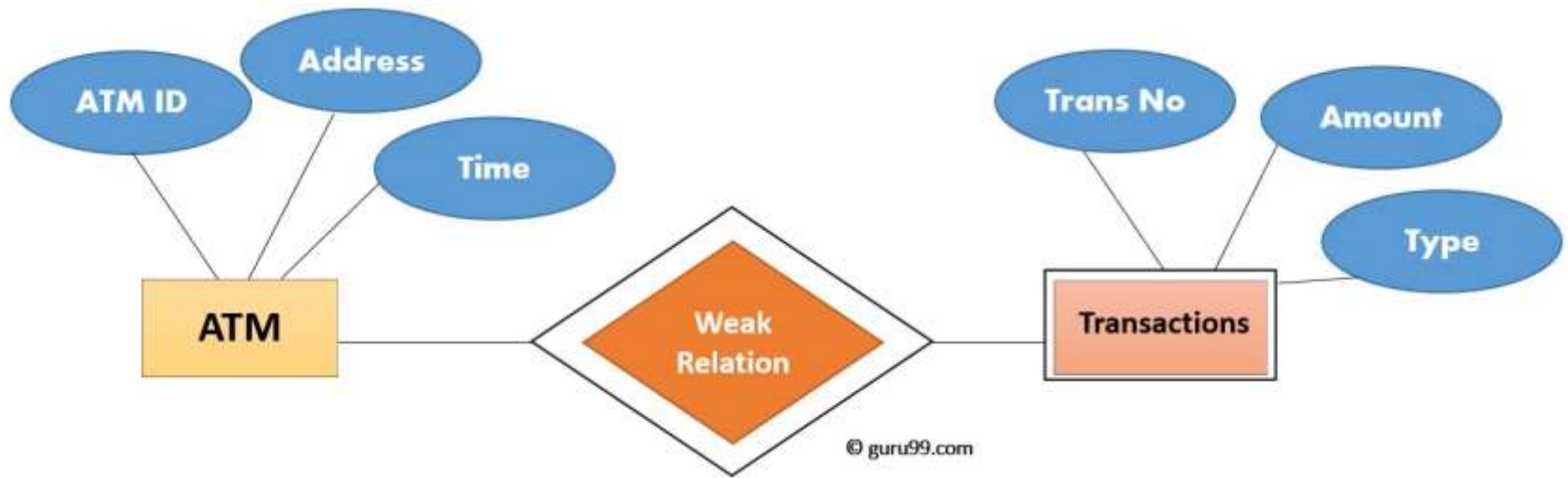


ER diagram has three main components:

- Weak Entity:
 - An entity that cannot be uniquely identified by its own attributes and relies on the relationship with other entity is called weak entity. The weak entity is represented by a double rectangle.
 - For example – a bank account cannot be uniquely identified without knowing the bank to which the account belongs, so bank account is a weak entity.



ER diagram has three main components:

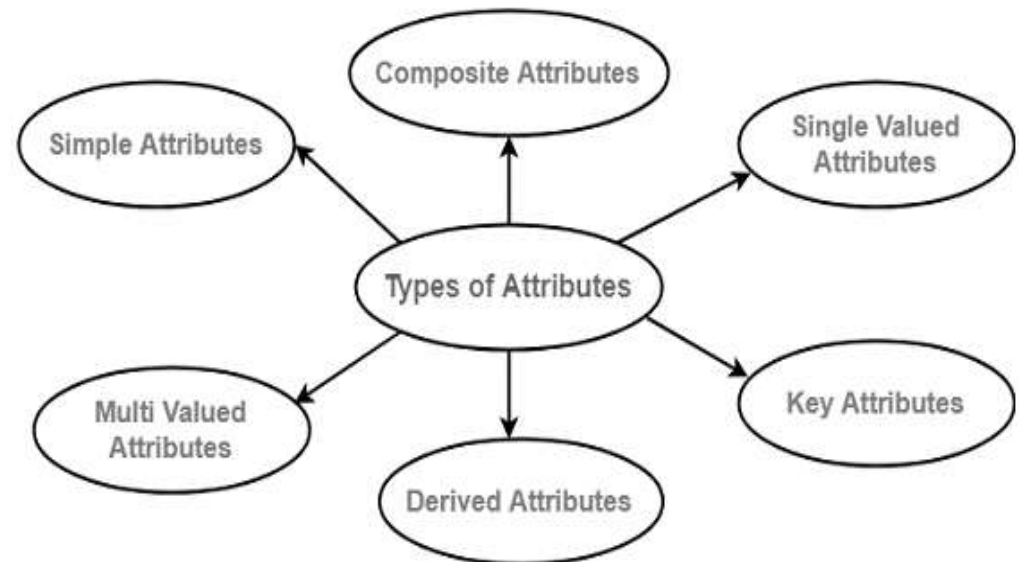


Differentiate between Weak and Strong entity

Strong Entity Set	Weak Entity Set
Strong entity set always has a primary key.	It does not have enough attributes to build a primary key.
It is represented by a rectangle symbol.	It is represented by a double rectangle symbol.
It contains a Primary key represented by the underline symbol.	It contains a Partial Key which is represented by a dashed underline symbol.
The member of a strong entity set is called as dominant entity set.	The member of a weak entity set called as a subordinate entity set.
Primary Key is one of its attributes which helps to identify its member.	In a weak entity set, it is a combination of primary key and partial key of the strong entity set.
In the ER diagram the relationship between two strong entity set shown by using a diamond symbol	The relationship between one strong and a weak entity set shown by using the double diamond symbol.

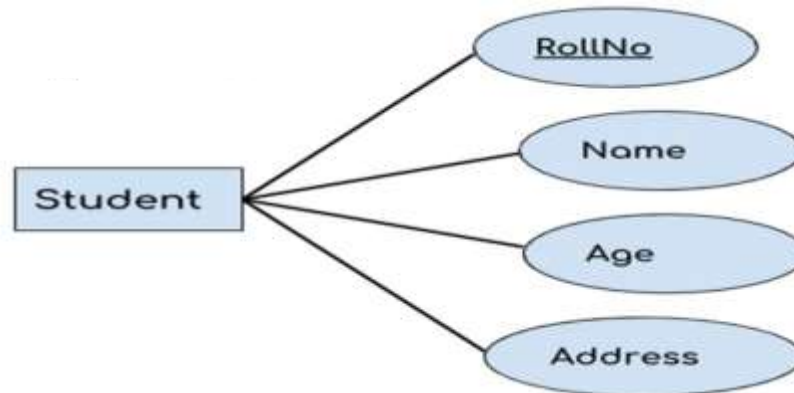
2. Attribute

- An attribute describes the property of an entity. An attribute is represented as Oval in an ER diagram.
- There are four types of attributes:
 - Key attribute
 - Composite attribute
 - Multivalued attribute
 - Derived attribute



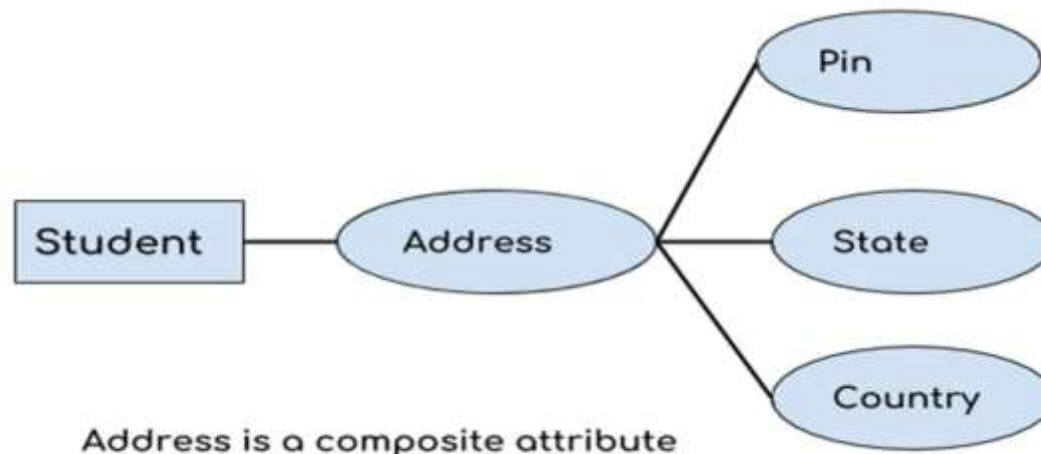
1. Key attribute

- A key attribute can uniquely identify an entity from an entity set.
- For example, student roll number can uniquely identify a student from a set of students.
- Key attribute is represented by oval same as other attributes however the text of key attribute is underlined.



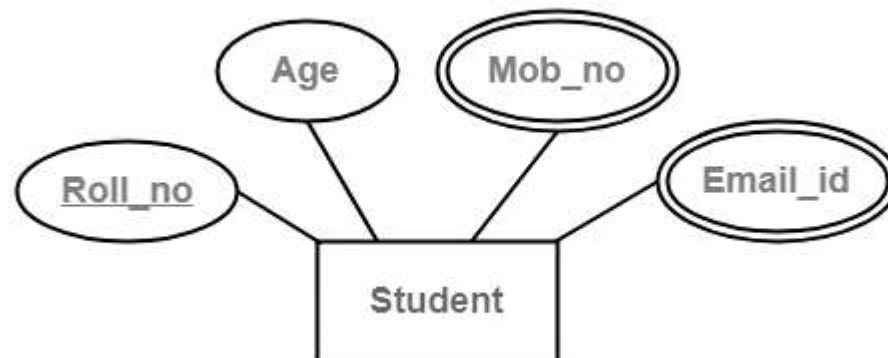
2. Composite Attribute

- An attribute that is a combination of other attributes is known as composite attribute. For example, In student entity, the student address is a composite attribute as an address is composed of other attributes such as pin code, state, country.



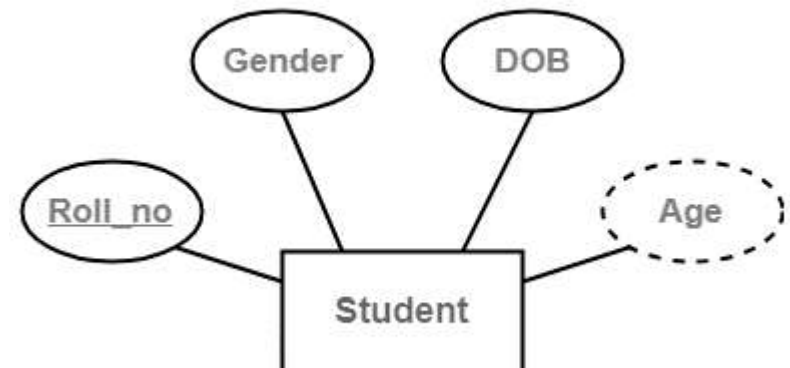
3. Multivalued attribute

- An attribute that can hold multiple values is known as multivalued attribute. It is represented with double ovals in an ER Diagram.
- For example – A person can have more than one phone numbers so the phone number attribute is multivalued.



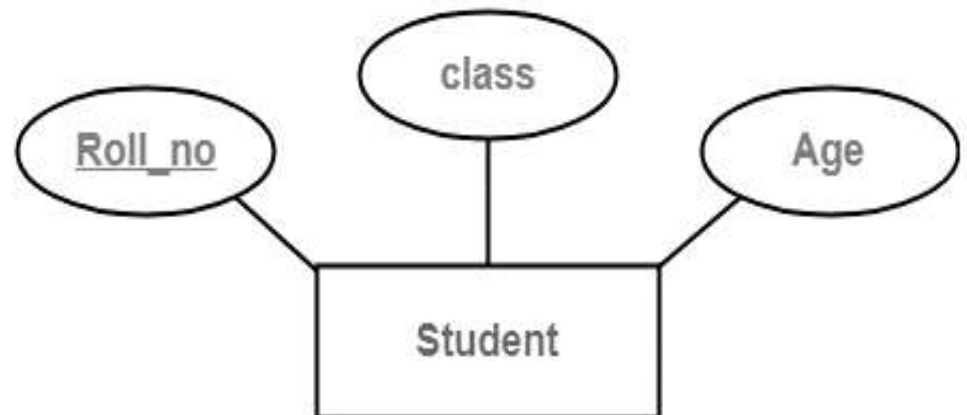
4. Derived attribute

- A derived attribute is one whose value is dynamic and derived from another attribute. It is represented by dashed oval in an ER Diagram.
- For example – Person age is a derived attribute as it changes over time and can be derived from another attribute (Date of birth).
- For example: Age can be derived from date of birth, where,
 - Age is the derived attribute.
 - DOB is the stored attribute.



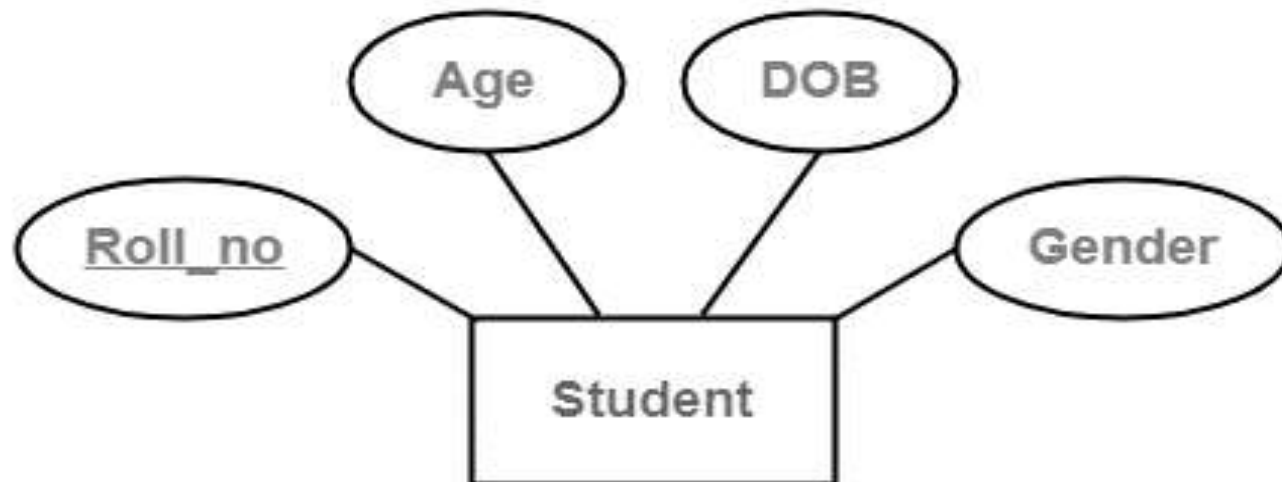
Simple or Atomic attribute

- Attributes that cannot be further subdivided are called atomic attributes.
- For example –
 - » Phone number
 - » PIN code

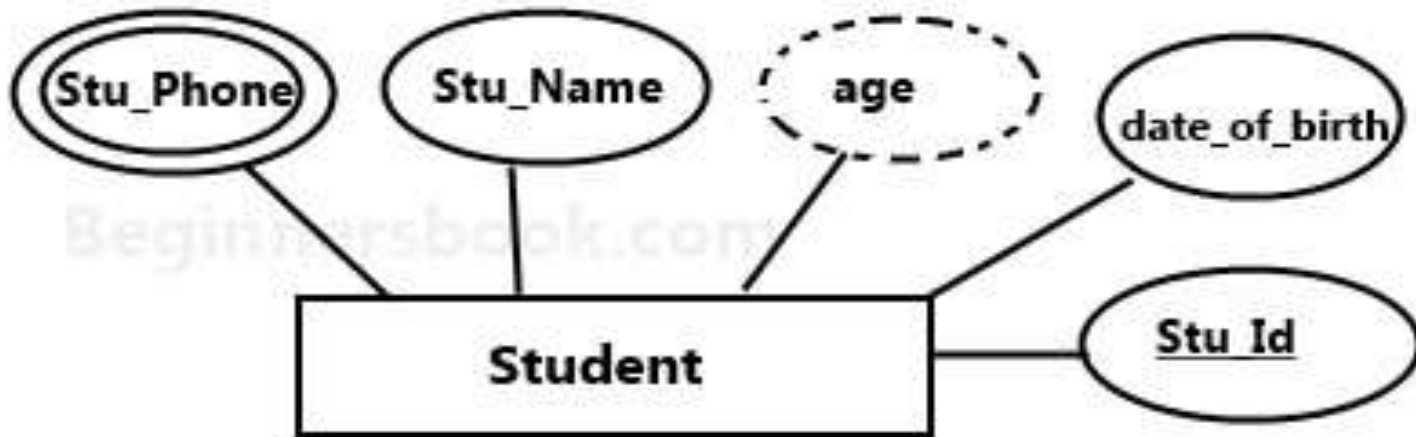


Single valued Attribute

- Attributes having a single value for a particular item is called a single valued attribute.
- For example: Room Number, The age of a student.

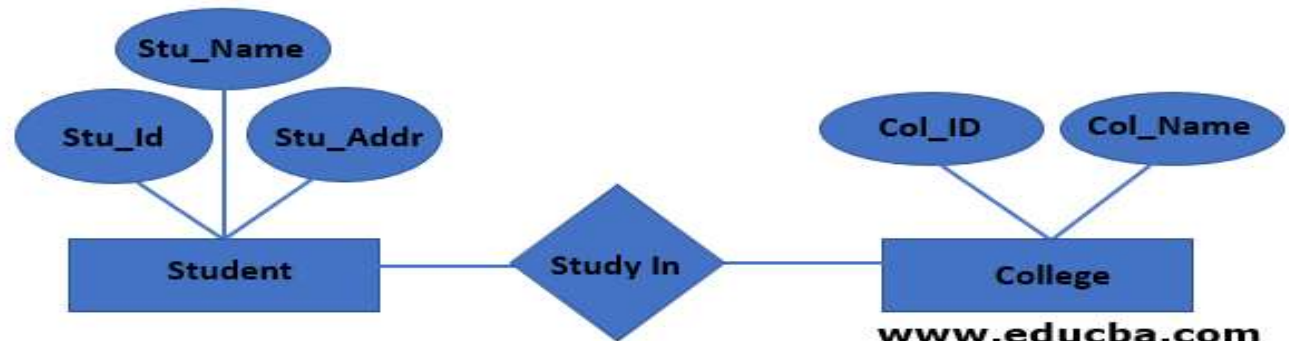


E-R diagram with multivalued and derived attributes:



Relationship

- A relationship in a DBMS, is primarily the way two or more data sets are linked. One dataset may be then termed as the Foreign key and the ones linked to it may be termed as the Primary Key. There may be multiple Foreign and Primary keys linked to each other.
- Relationships allow the datasets to share and store the data in separate tables. They also help link disparate data with each other.



Types of Relationship in DBMS

- There are three types of relationships that can be found in DBMS:
 - One to One
 - One to Many
 - Many to Many

One to One

- This relationship tells us that a single record in Table A is related to a single record in Table B. And vice versa.
- It is used to create a relationship between two tables in which a single row of the first table can only be related to one and only one records of a second table. Similarly, the row of a second table can also be related to anyone row of the first table.

Customer **(Table A)**

Peter
Susan McLain
Rosales
Bob Jones
Sue Williams
Brown
Howard
Taylor

Order **(Table B)**

Ord102
Ord103
Ord105
Ord107
Ord108
Ord120
Ord106
Ord100



One to One

- **For example:** We have to create a table where details of our **name, age, email, address, telephone number and password** needs to be stored. Now, all of this are sensitive information and we would not want to have them bundled together in a single table. Hence, to create a greater security around it, we create a separate table for the password with different access control, so that only a selected few will have access to it.

One to Many

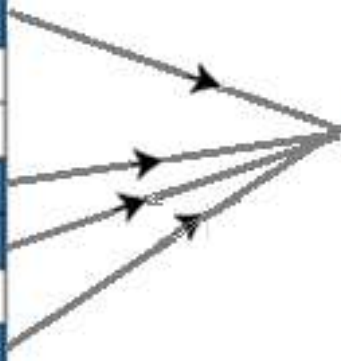
- Representation of **many to one** relational database

Student(**Table A**)

Jasmine
James
Margret
Nicolas
Nupur
Rosy
Alexandra
Morris

Teacher(**Table B**)

Amy Smith
Edna francis
Sarah Norris
Abigail Marlow
Elizabeth
Vaughn
Helen Smith
Jennie Hall

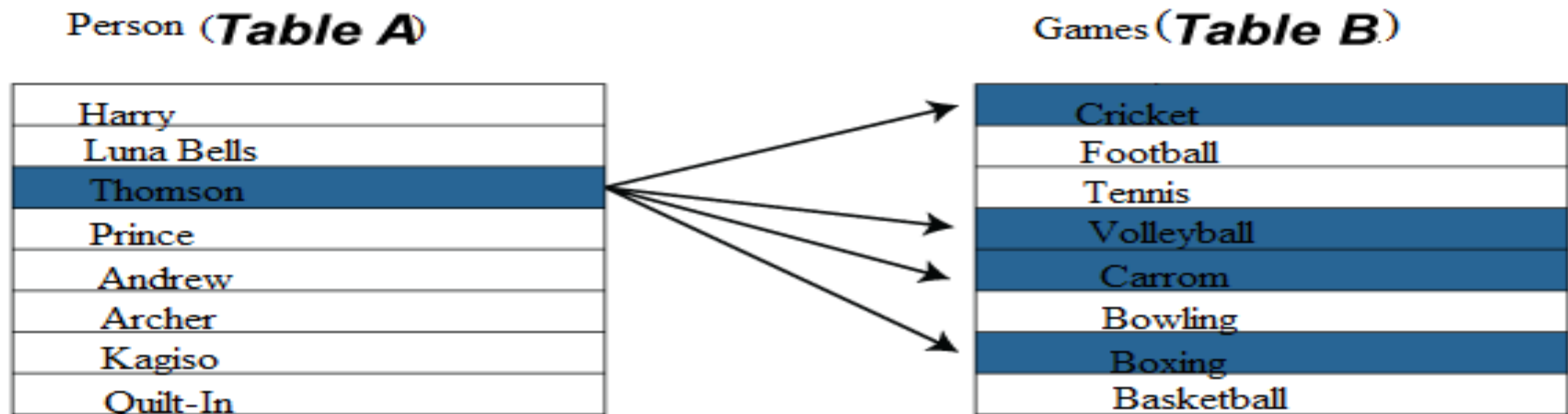


One to Many

- This type of relationship in a Database Management System (DBMS) is the most common type of relationship. This entails one data in Table A to have links to multiple data in Table B.
- However, a single data in Table B, will have link to a single data in Table A. One good example to showcase such a relationship is through a bank account. One individual may have multiple bank accounts, but each bank account will be linked to one specific owner or account holder only (provided we are not talking about a joint bank account)

One to Many

- It is used to create a relationship between two tables. Any single rows of the first table can be related to one or more rows of the second tables, but the rows of second tables can only relate to the only row in the first table. It is also known as a **many to one** relationship.
- Representation of **One to Many** relational databases:



Many to Many

- It is **many to many** relationships that create a relationship between two tables. Each record of the first table can relate to any records (or no records) in the second table. Similarly, each record of the second table can also relate to more than one record of the first table. It is also represented an **N:N** relationship.
- For example, there are **many** people involved in each **project**, and every person can involve more than one project.



Many to Many

- Consider two table A and B. Each data in Table A is linked to all the data in Table B and vice versa. A great example would be linking a student table to the courses table. Table A has the name of all the students while Table B has the name of all the courses. A particular student may enrol oneself to more than one course whilst a course may have more than one student. Hence, it becomes a many to many relationships.

What is a cardinality database

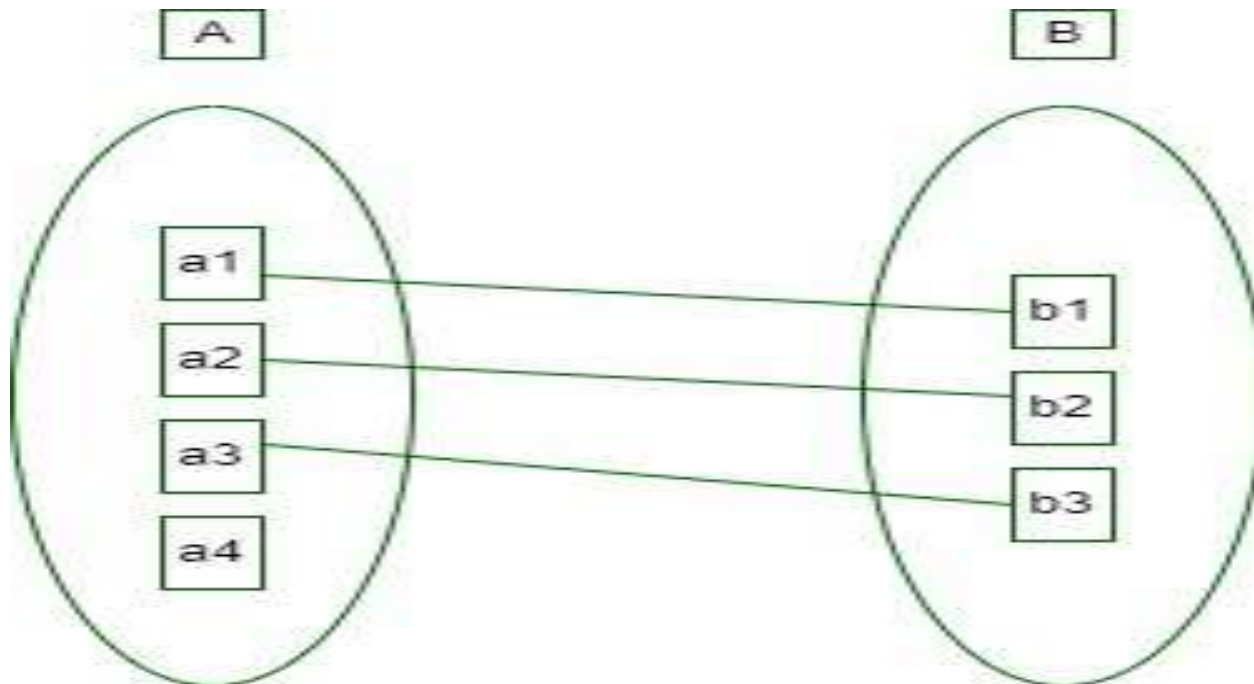
- Cardinality is a mathematical term that refers to the number of elements in a given set. Database administrators may use cardinality to count tables and values. In a database, cardinality usually represents the relationship between the data in two different tables by highlighting how many times a specific entity occurs compared to another.
- For example, the database of an auto repair shop may show that a mechanic works with multiple customers every day. This means that the relationship between the mechanic entity and the customer entity is one mechanic to many customers.

Types of cardinality in databases

- Types of cardinality in between tables are:
 - one-to-one
 - one-to-many
 - many-to-one
 - many-to-many

One-to-one

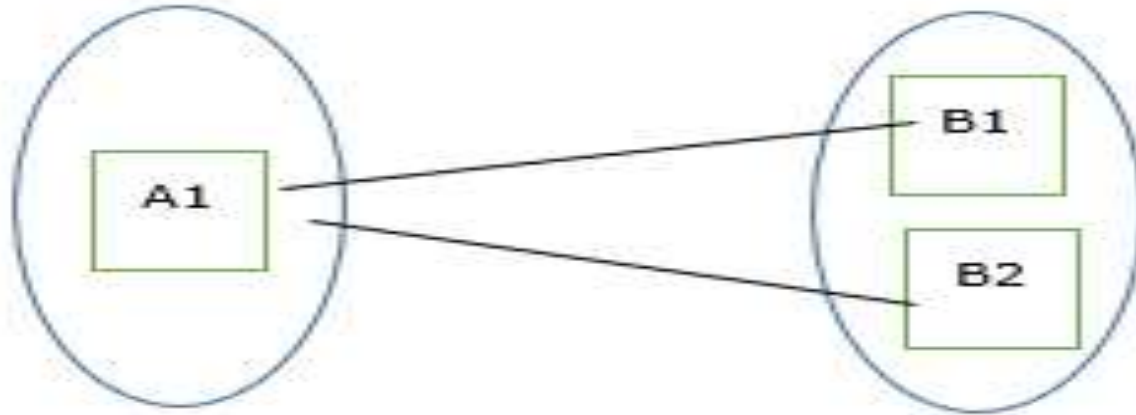
- One-to-one: In this type of cardinality mapping, an entity in A is connected to at most one entity in B. Or we can say that a unit or item in B is connected to at most one unit or item in A.



One-to-many relationship

- An entity set A is associated with any number of entities in B with a possibility of zero and entity in B is associated with at most one entity in A
- A one-to-many (1:Many) relationship describes a situation where one occurrence in an entity relates to many occurrences in another entity. You might see this type of cardinality in a database if you're working with one row in a specific table that relates to multiple rows in a different table.

One-to-many relationship



Example

Given below is an example of the one-to-many relationship in the mapping cardinality. Here, one department has many faculties.

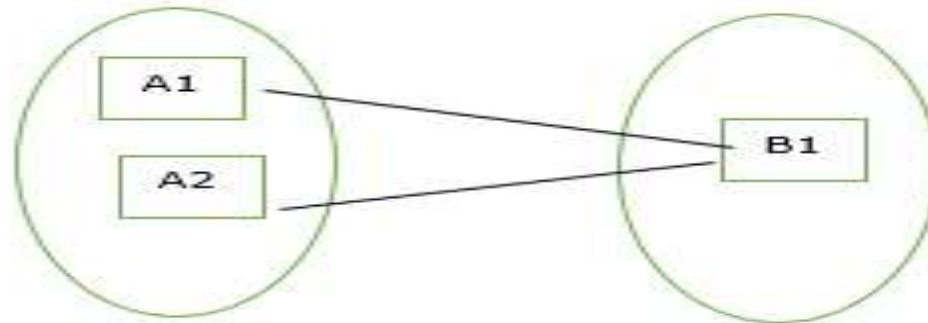


- Example:
- In a particular hospital, the surgeon department has one head of department. They both serve one-to-one relationships.

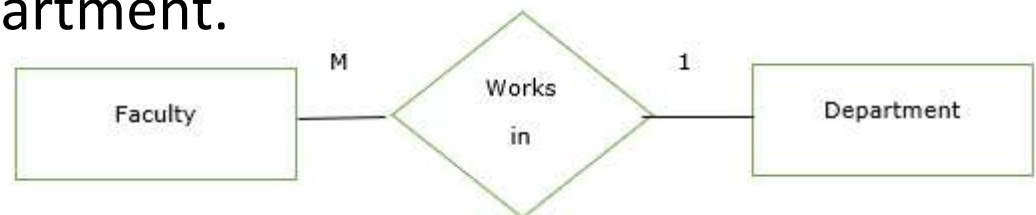


Many-to-one relationship

- An entity set A is associated with at most one entity in B and an entity set in B can be associated with any number of entities in A with a possibility of zero.



many faculties work in one department.



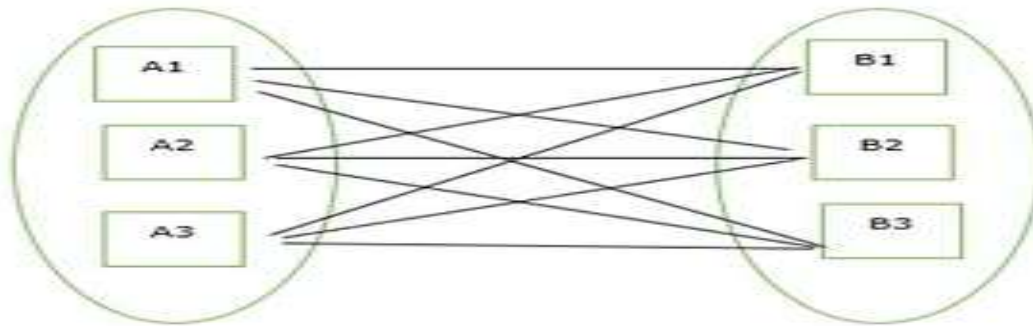
Many-to-many relationship

- A many-to-many (M:M) relationship describes a situation where multiple occurrences in one entity relate to multiple occurrences in another entity. You might see this type of cardinality in a database if you're working with several rows in a specific table that relate to several rows in another table.

***For Example :** A book retailer may use the many-to-many relationship model to manage their online database. They might include a list of book titles in one table and a list of author names in another table. Many of the authors may have multiple books and some of the authors may have even co-*

Many-to-many relationship

- An entity in A is associated with many entities of B and an entity in B is associated with many entities of A.
- Many to many=many to one + one to many



many employees work on many projects.

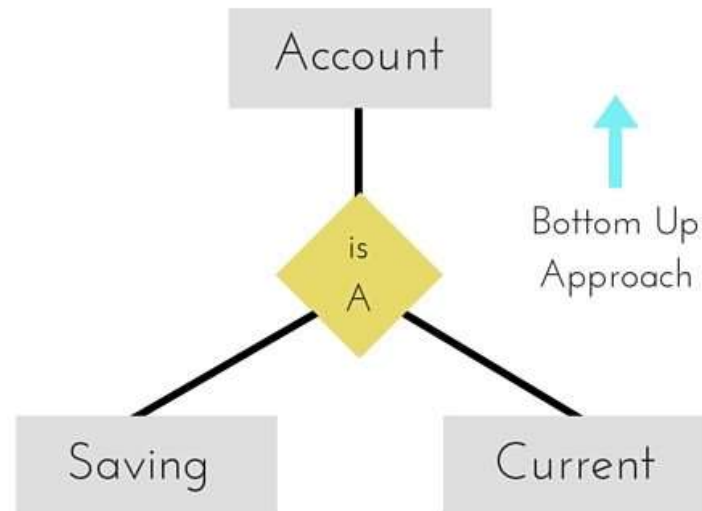


Generalization in DBMS

- Suppose you are storing details of cars and bikes in your database and you want to simplify the values by storing their common attributes in a specific entity, So in that case, you can generalize the database by merging the common attributes of cars and bikes.
- **Generalization** is a bottom-up approach in which two lower level entities combine to form a higher level entity. In generalization, the higher level entity can also combine with other lower level entities to make further higher level entity.

Generalization in DBMS

- It's more like Superclass and Subclass system, but the only difference is the approach, which is bottom-up. Hence, entities are combined to form a more generalised entity, in other words, sub-classes are combined to form a super-class.

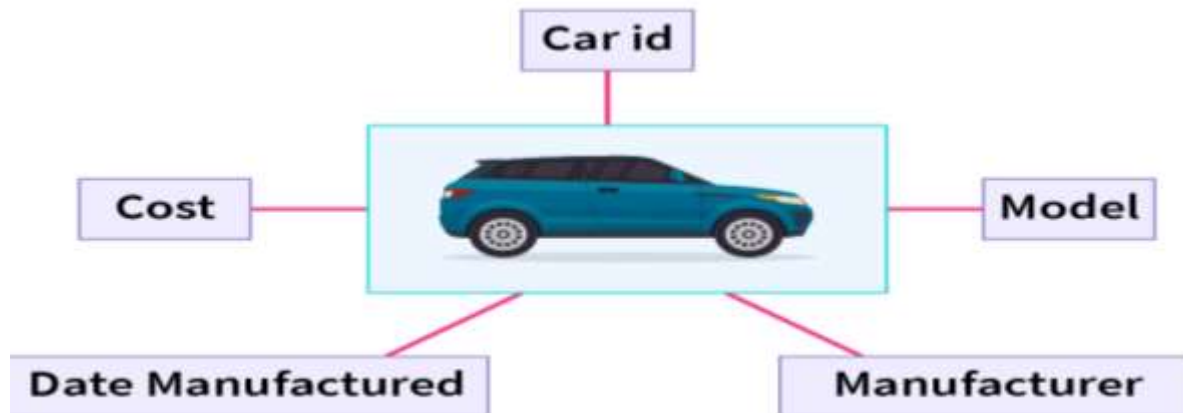


Generalization in DBMS

- Generalization is a process in which a new entity is formed using the common attributes of two or more entities. Generalization simplifies the ER diagram by clubbing the common attributes together. Below are some characteristics of generalization:
- Generalization follows the bottom-up approach.
- It generalizes or simplifies the entities.
- Higher-level entities can also be combined with lower-level entities.

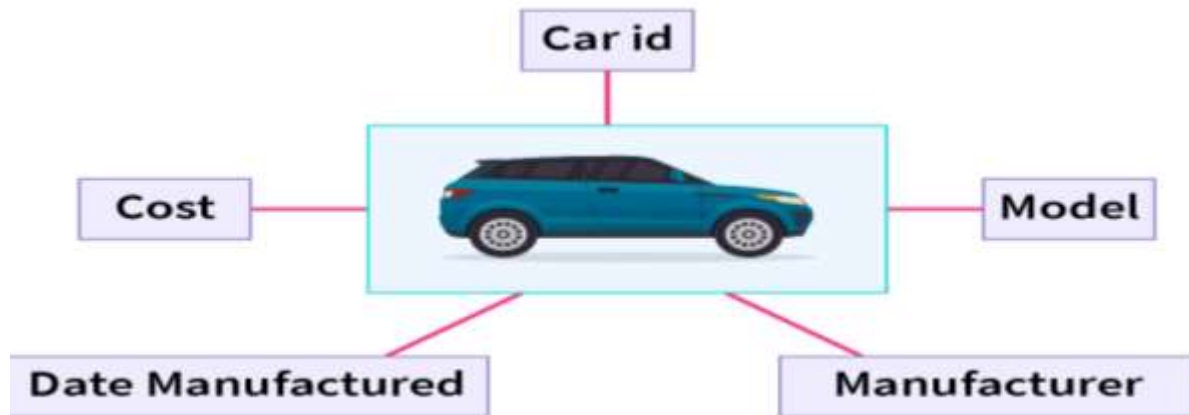
Generalization in DBMS

- **Entities Before Generalization**
- Below will be the entities before generalization:
- **Car Entity:** Car entity can have attributes like car_ID, DateManufactured, Cost, Model, Manufacturer which will be generalized further.



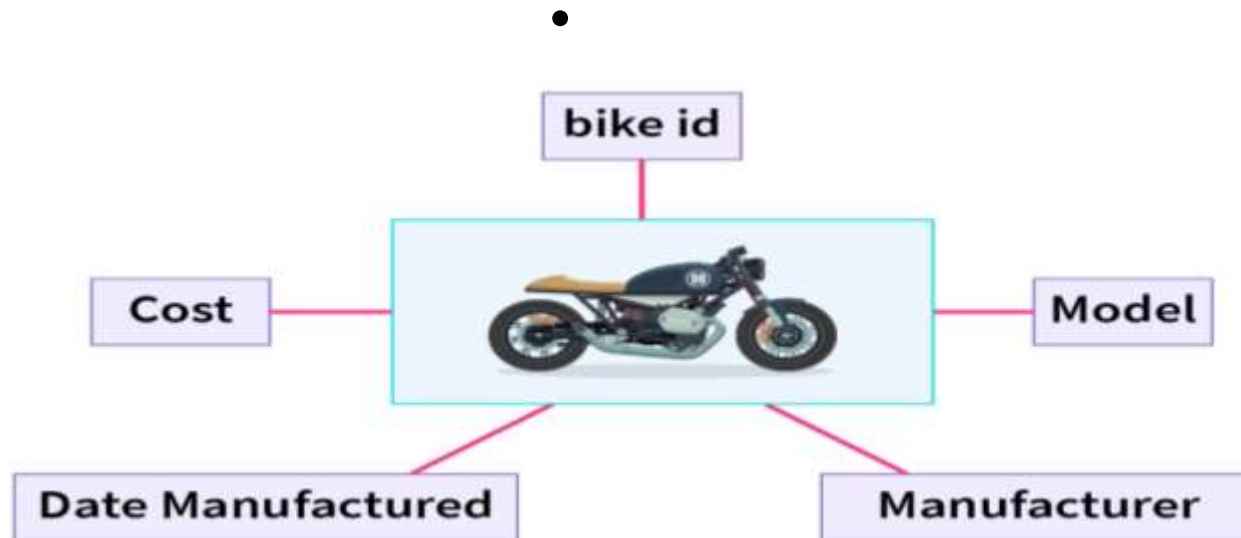
Generalization in DBMS

- **Entities Before Generalization**
- Below will be the entities before generalization:
- **Car Entity:** Car entity can have attributes like car_ID, DateManufactured, Cost, Model, Manufacturer which will be generalized further.



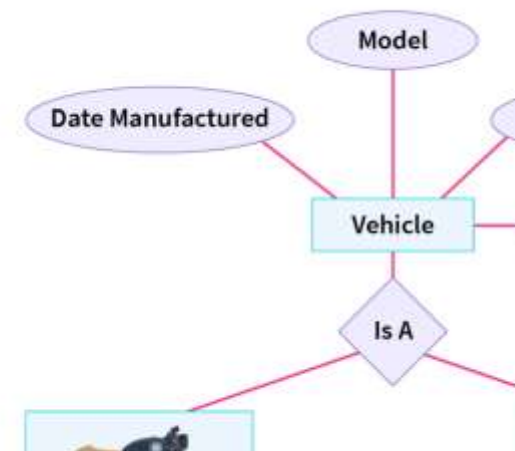
Generalization in DBMS

- **Bike Entity:** Bike entity can have attributes like bike_ID, DateManufactured, Cost, Model, Manufacturer which will be generalized further.

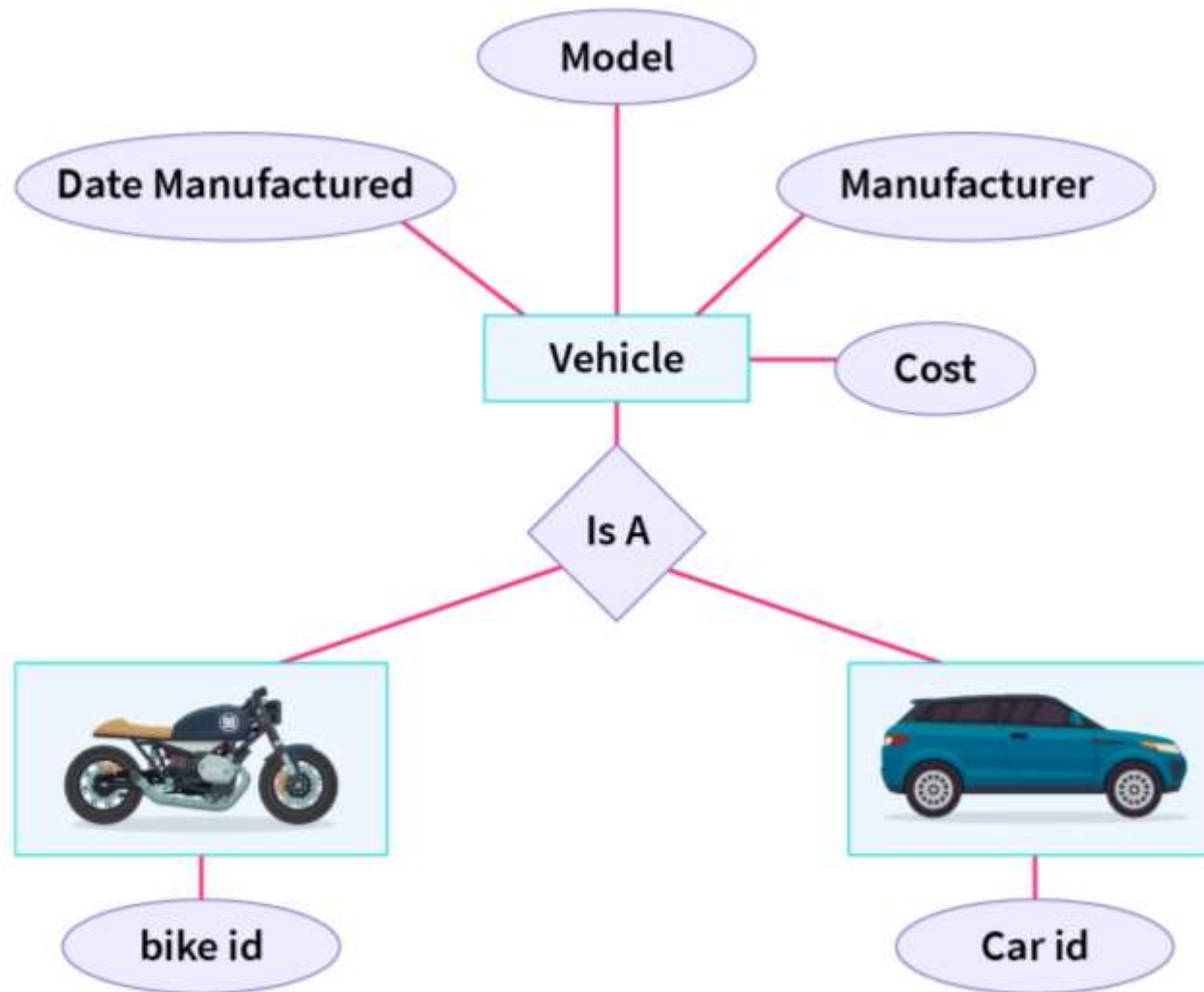


ER Diagram after Generalization

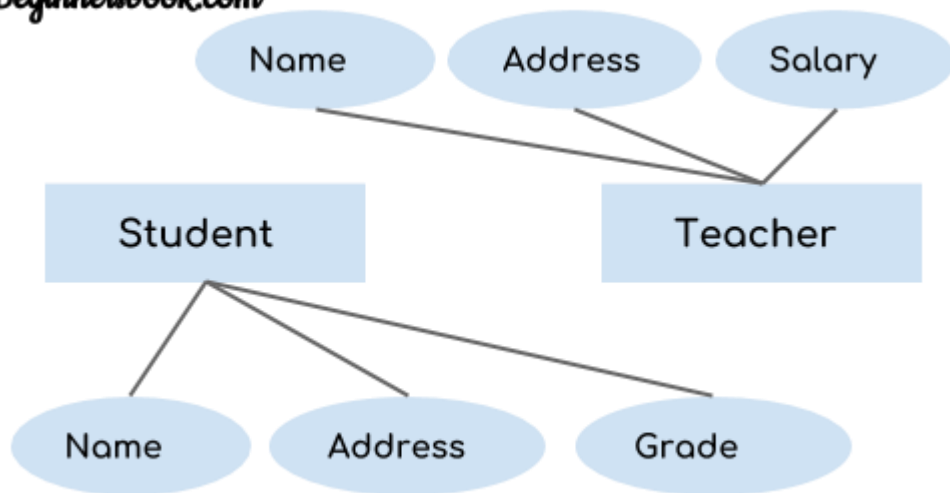
- We can see that for cars and bikes some attributes like *DateManufactured*, *Cost*, *Model*, *Manufacturer* are the same so we can generalize them into an entity called **Vehicle**.
- The **vehicle** entity will be having these common attributes and the bike and car entity will be related to them using "Is A" relationship table, So ER diagram can be



ER Diagram after Generalization

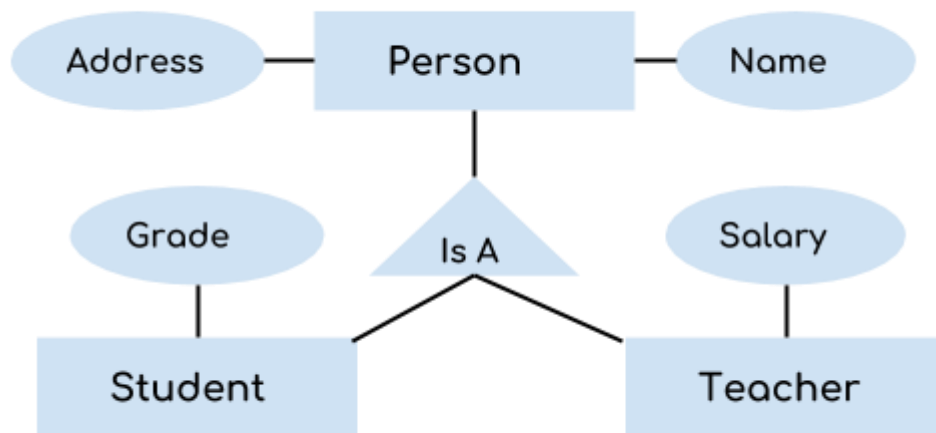


Beginnersbook.com

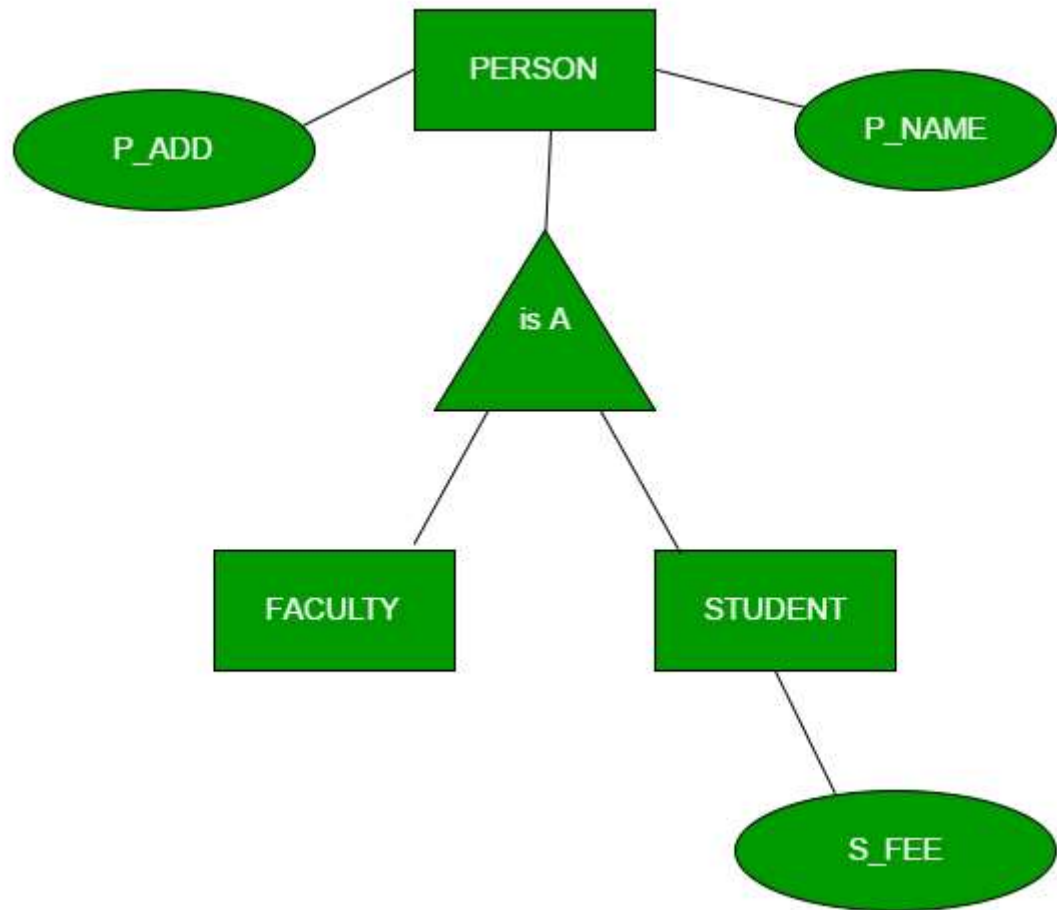


Before Generalization

Beginnersbook.com

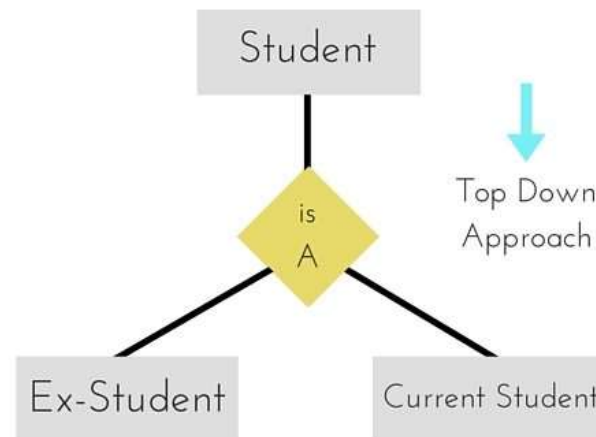


Generalization



Specialization

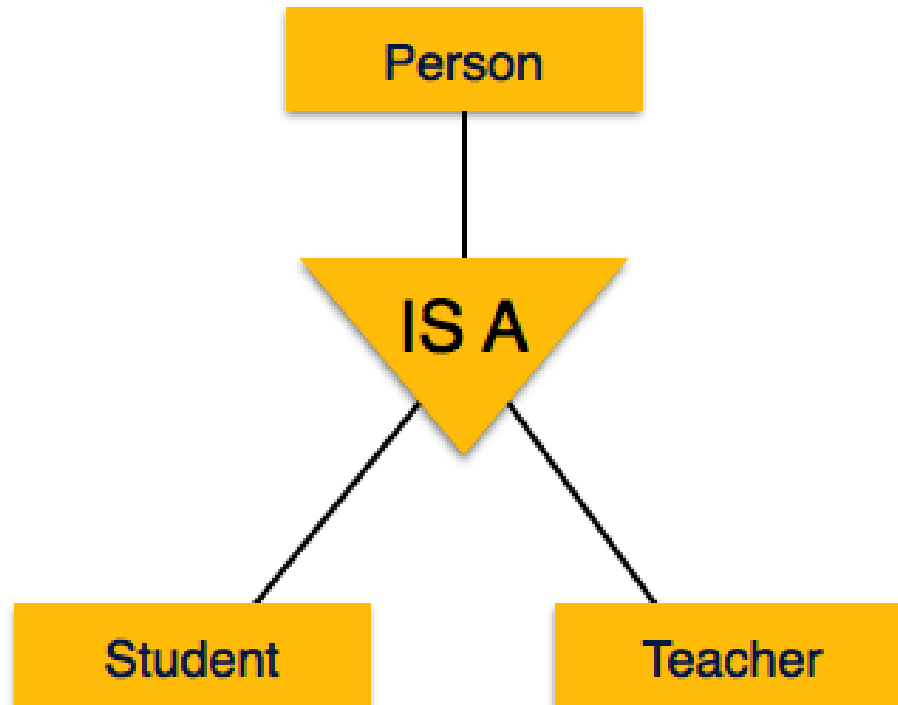
- **Specialization** is opposite to Generalization. It is a top-down approach in which one higher level entity can be broken down into two lower level entity. In specialization, a higher level entity may not have any lower-level entity sets, it's possible.



Specialization

- Specialization is the opposite of generalization. In specialization, a group of entities is divided into sub-groups based on their characteristics. Take a group 'Person' for example. A person has name, date of birth, gender, etc. These properties are common in all persons, human beings. But in a company, persons can be identified as employee, employer, customer, or vendor, based on what role they play in the company.

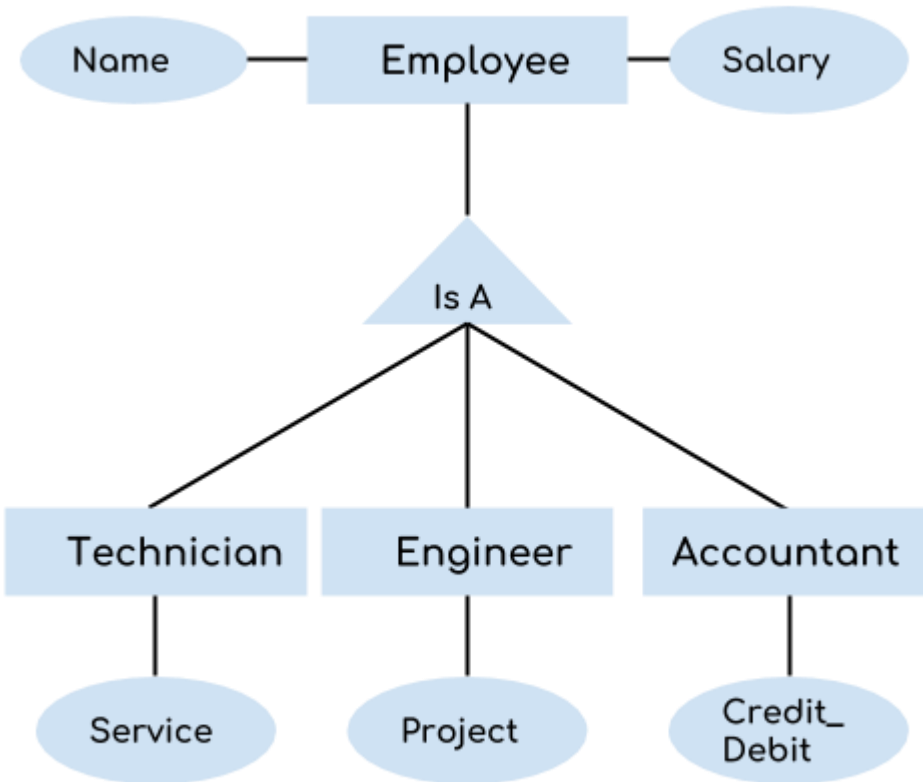
Specialization



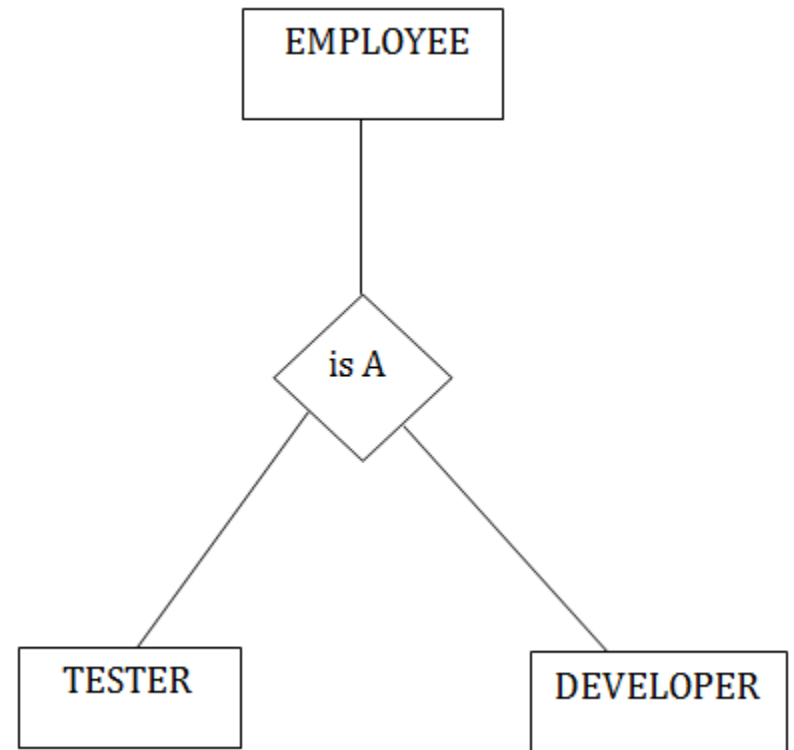
Similarly, in a school database, persons can be specialized as teacher, student, or a staff, based on what role they play in school as entities.

Example

Beginnerbook.com



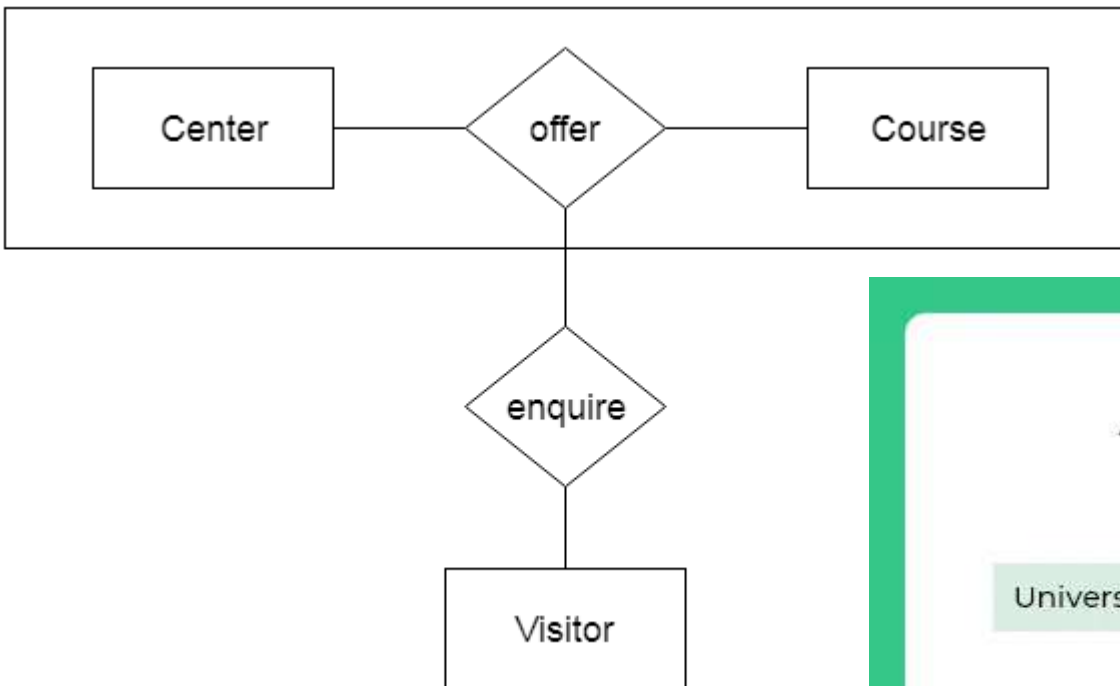
Specialization



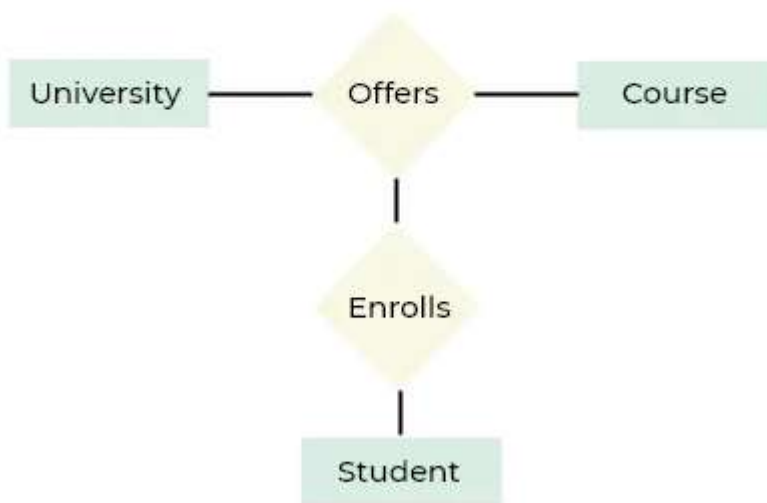
Aggregation in DBMS

- Aggregation in DBMS (Database Management System) is a process of combining two or more entities to form a more meaningful new entity. This Aggregation process is done when the entities don't make sense on their own without applying the aggregation process.
- **For example:** Center entity offers the Course entity act as a single entity in the relationship which is in a relationship with another entity visitor. In the real world, if a visitor visits a coaching center then he will never enquiry about the Course only or just about the Center instead he will ask the enquiry about both.

Example



Aggregation in DBMS



Why aggregation

- Aggregation is a process of compiling information on an object thereby abstracting higher-level object
- In SQL we need to find the sum of salaries of all the employees working in an organization or to find the highest-paid employee from all branches of the organization etc.
- when using data in the form of numerical values, the following operations can be used to perform DBMS aggregation:
- **Average (AVG):** This function provides the mean or average of the data values.
- **Sum:** This provides a total value after the data values have been added.

Why aggregation

- **Count:** This provides the number of records.
- **Maximum (Max):** This function provides the maximum value of a given set of data.
- **Minimum (Min):** This provides the minimum value of a given set of data.
- **Standard deviation (std dev):** This provides the dispersion or variation of the sets of data. Let's take a simple example of a database of student marks. If the standard deviation is high, it means the average is obtained by lower number of students than usual, and the lowest and highest marks are higher.

Relational data model

- **Relational Model (RM)** represents the database as a collection of relations. A relation is nothing but a table of values. Every row in the table represents a collection of related data values. These rows in the table denote a real-world entity or relationship.
- The table name and column names are helpful to interpret the meaning of values in each row. The data are represented as a set of relations. In the relational model, data are stored as tables

Relational Model Concepts in DBMS

- **Attribute:** Each column in a Table. Attributes are the properties which define a relation. e.g., Student_Rollno, NAME,etc.
- **Tables** – In the Relational model the, relations are saved in the table format. It is stored along with its entities. A table has two properties rows and columns. Rows represent records and columns represent attributes

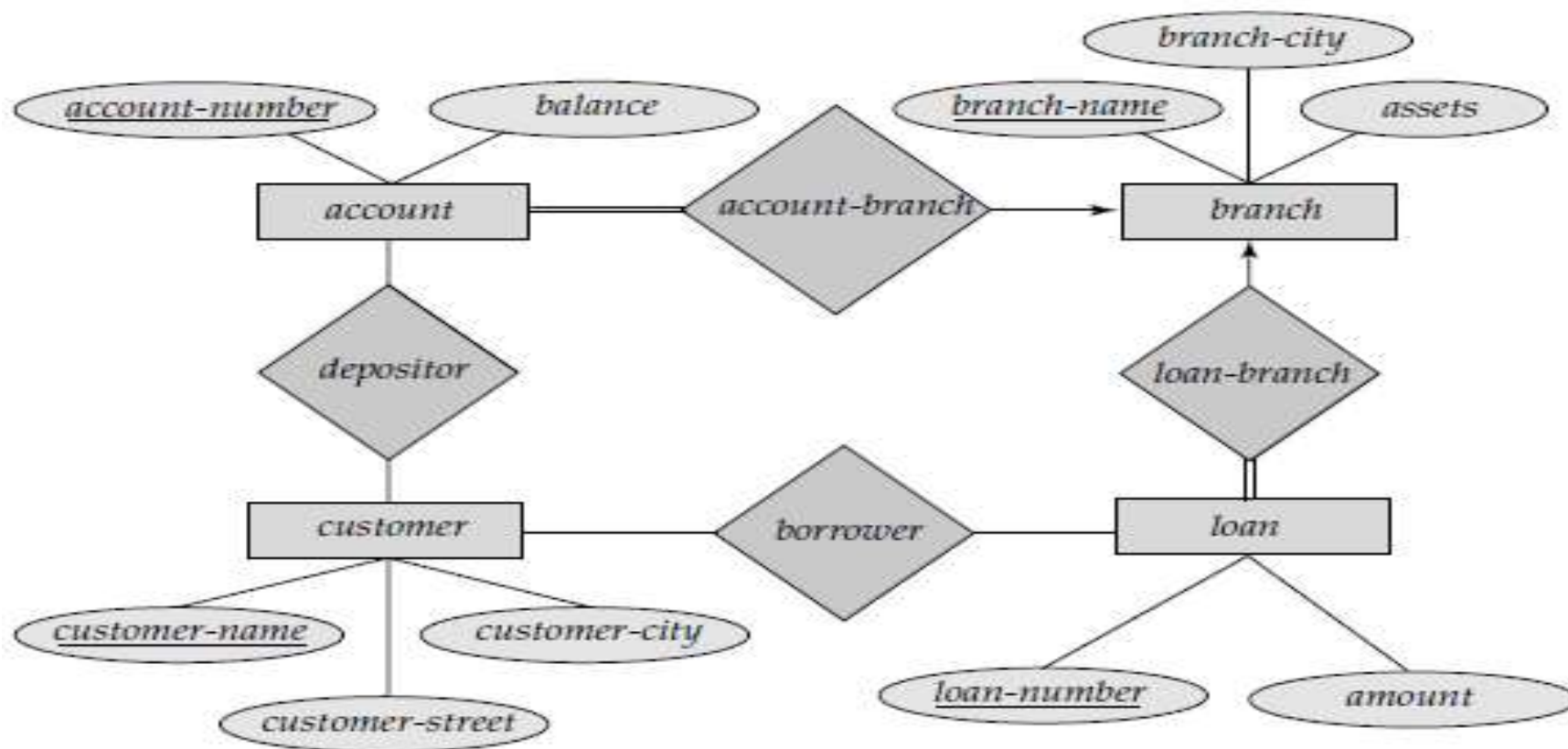
Relational Model Concepts in DBMS

- **Tuple** – It is nothing but a single row of a table, which contains a single record.
- **Relation Schema:** A relation schema represents the name of the relation with its attributes.
- **Degree:** The total number of attributes which in the relation is called the degree of the relation.
- **Cardinality:** Total number of rows present in the Table.

Relational Model Concepts in DBMS

- **Column:** The column represents the set of values for a specific attribute.
- **Relation instance** – Relation instance is a finite set of tuples in the RDBMS system. Relation instances never have duplicate tuples.
- **Relation key** – Every row has one, two or multiple attributes, which is called relation key.
- **Attribute domain** – Every attribute has some pre-defined value and scope which is known as attribute domain

Types of keys

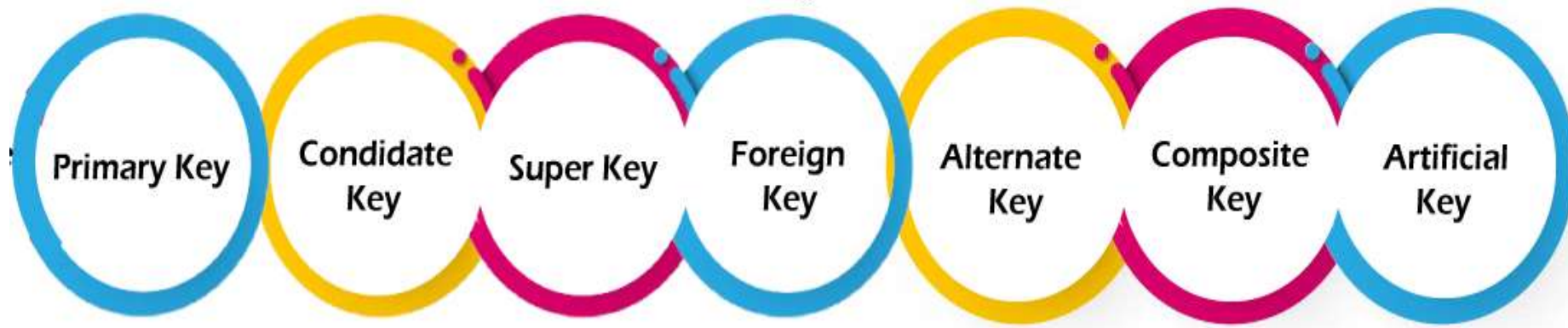


Branch-schema, $\{branch \ customer \ name\}$ and $\{branch-name, \ branch-city\}$ are both superkeys. $\{branch-name, \ branch-city\}$ is not a candidate key, because $\{branch-name\}$ is a subset of $\{branch-name, \ branchcity\}$ and $\{branch-name\}$ itself is a superkey.

Types of keys

KEYS in DBMS is an attribute or set of attributes which helps you to identify a row(tuple) in a relation(table). They allow you to find the relation between two tables. Keys help you uniquely identify a row in a table by a combination of one or more columns in that table. Key is also helpful for finding unique record or row from the table. Database key is also helpful for finding unique record or row from the table.

Keys



Super Key

- Super key is an attribute set that can uniquely identify a tuple. A super key is a superset of a candidate key.
- A super key is a group of single or multiple keys which identifies rows in a table.
- A super key is a set of attributes that can identify each tuple uniquely in the given relation.
- A super key is not restricted to have any specific number of attributes.
- Thus, a super key may consist of any number of attributes.

Super Key

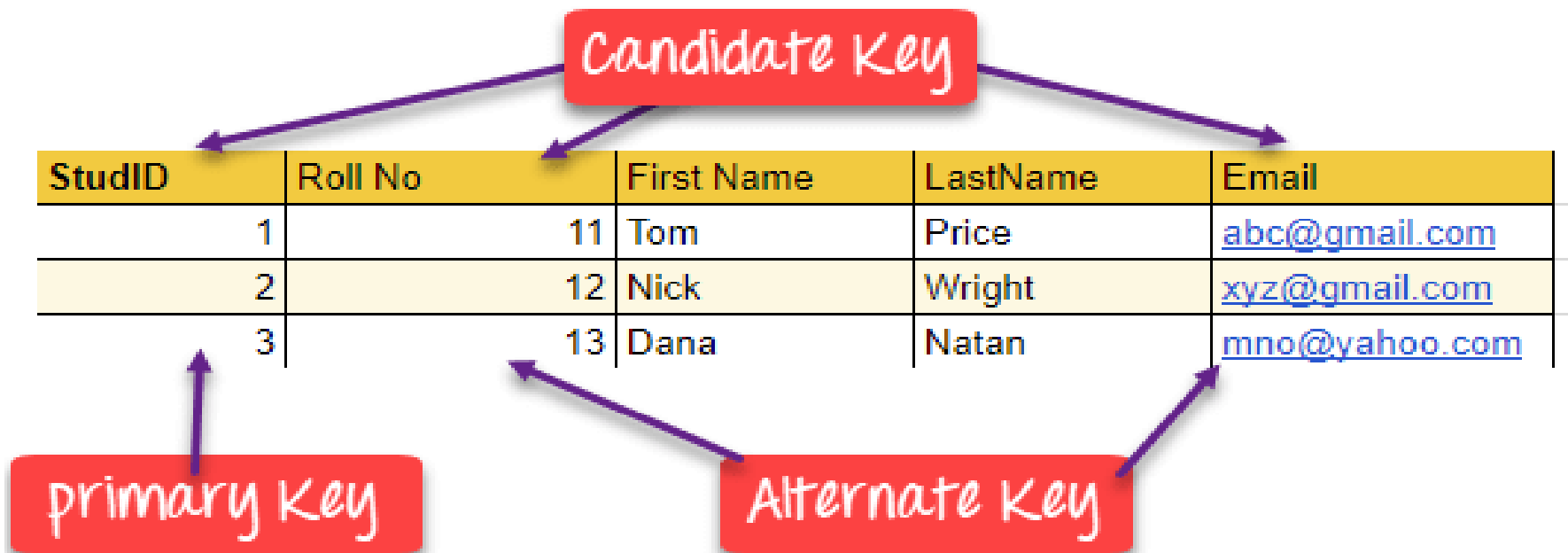
- **Example-**
- Consider the following Student schema-
- **Student (roll , name , sex , age , address , class , section)**
- Given below are the examples of super keys since each set can uniquely identify each student in the Student table-
- Given below are the examples of super keys since each set can uniquely identify each student in the Student table-
 - (roll , name , sex , age , address , class , section)
 - (class , section , roll)
 - (class , section , roll , sex)
 - (name , address)

Candidate Key

- **CANDIDATE KEY** in SQL is a set of attributes that uniquely identify tuples in a table. Candidate Key is a super key with no repeated attributes. The Primary key should be selected from the candidate keys. Every table must have at least a single candidate key. A table can have multiple candidate keys but only a single primary key.
- **Properties of Candidate key:**
 - It must contain unique values
 - Candidate key in SQL may have multiple attributes
 - Must not contain null values
 - It should contain minimum fields to ensure uniqueness
 - Uniquely identify each record in a table

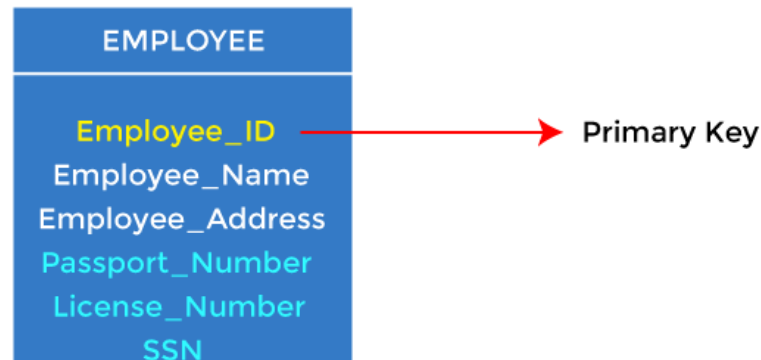
Candidate Key

- Candidate key Example: In the given table Stud ID, Roll No, and email are candidate keys which help us to uniquely identify the student record in the table.



Primary Key-

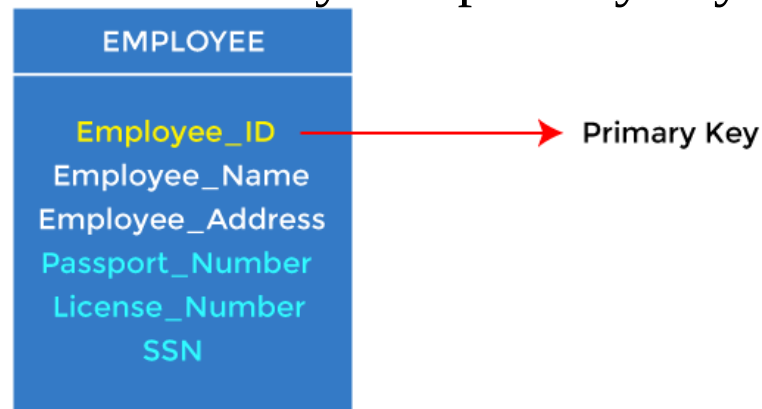
- It is the first key used to identify one and only one instance of an entity uniquely. An entity can contain multiple keys, as we saw in the PERSON table. The key which is most suitable from those lists becomes a primary key.
- In the EMPLOYEE table, ID can be the primary key since it is unique for each employee. In the EMPLOYEE table, we can even select License_Number and Passport_Number as primary keys since they are also unique.
- For each entity, the primary key selection is based on requirements and developers.



Primary Key-

- **NOTES-**

- The value of primary key can never be NULL.
- The value of primary key must always be unique.
- The values of primary key can never be changed i.e. no updation is possible.
- The value of primary key must be assigned when inserting a record.
- A relation is allowed to have only one primary key.



What is the Alternate key?

- **ALTERNATE KEYS** is a column or group of columns in a table that uniquely identify every row in that table. A table can have multiple choices for a primary key but only one can be set as the primary key. All the keys which are not primary key are called an Alternate Key.
- **Example:**
- In this table, StudID, Roll No, Email are qualified to become a primary key. But since StudID is the primary key, Roll No, Email becomes the alternative key.

StudID	Roll No	First Name	LastName	Email
1	11	Tom	Price	abc@gmail.com
2	12	Nick	Wright	xyz@gmail.com
3	13	Dana	Natan	mno@yahoo.com

What is the Foreign key?

- **FOREIGN KEY** is a column that creates a relationship between two tables. The purpose of Foreign keys is to maintain data integrity and allow navigation between two different instances of an entity. It acts as a cross-reference between two tables as it references the primary key of another table.

DeptCode	DeptName
001	Science
002	English
005	Computer

Tacher ID	Fname	Lname
B002	David	Warner
B017	Sara	Joseph
B009	Mike	Brunton

What is the Foreign key?

- we have two table, teacher and department in a school. However, there is no way to see which search work in which department.
- In this table, adding the foreign key in Deptcode to the Teacher name, we can create a relationship between the two tables.

This concept is also known as Referential Integrity.

Teacher ID	DeptCode	Fname	Lname
B002	002	David	Warner
B017	002	Sara	Joseph
B009	001	Mike	Brunton

What is the Compound key?

- **COMPOUND KEY** has two or more attributes that allow you to uniquely recognize a specific record. It is possible that each column may not be unique by itself within the database.
- when combined with the other column or columns the combination of composite keys become unique. The purpose of the compound key in database is to uniquely identify each record in the table.

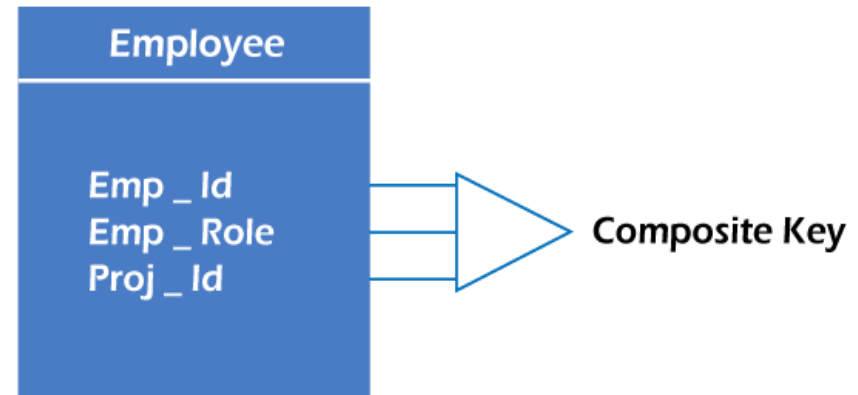
- In this example, OrderNo and ProductID can't be a primary key as it does not uniquely identify a record. However, a compound key of Order ID and Product ID could be used as it uniquely identified each record.

OrderNo	PorductID	Product Name	Quantity
B005	JAP102459	Mouse	5
B005	DKT321573	USB	10
B005	OMG446789	LCD Monitor	20
B004	DKT321573	USB	15
B002	OMG446789	Laser Printer	3

What is the Composite key?

- **COMPOSITE KEY** is a combination of two or more columns that uniquely identify rows in a table. The combination of columns guarantees uniqueness, though individually uniqueness is not guaranteed. Hence, they are combined to uniquely identify records in a table.

Whenever a primary key consists of more than one attribute, it is known as a composite key. This key is also known as Concatenated Key.



What is a Surrogate key?

- **SURROGATE KEYS** is An artificial key which aims to uniquely identify each record is called a surrogate key. This kind of partial key in dbms is unique because it is created when you don't have any natural primary key. They do not lend any meaning to the data in the table. Surrogate key in DBMS is usually an integer. A surrogate key is a value generated right before the record is inserted into a table.

What is a Surrogate key?

Fname	Lastname	Start Time	End Time
Anne	Smith	09:00	18:00
Jack	Francis	08:00	17:00
Anna	McLean	11:00	20:00
Shown	Willam	14:00	23:00

shift timings of the different employee. In this example, a surrogate key is needed to uniquely identify each employee.

Surrogate keys in sql are allowed when
No property has the parameter of the primary
key.

In the table when the primary key is too big or
complicated.

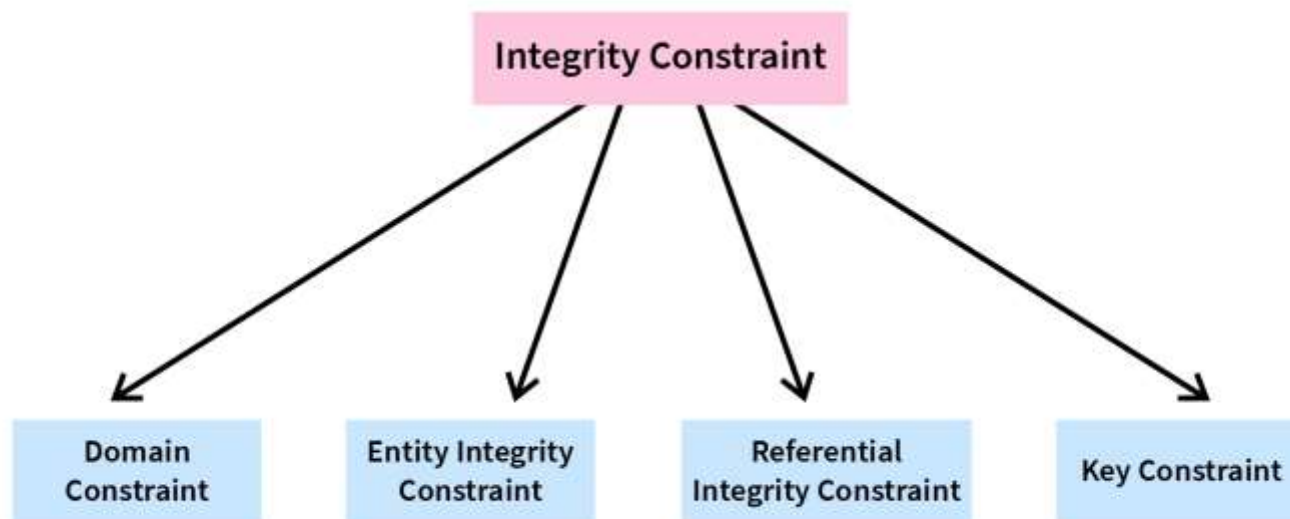
Difference Between Primary key & Foreign key

Primary Key	Foreign Key
Helps you to uniquely identify a record in the table.	It is a field in the table that is the primary key of another table.
Primary Key never accept null values.	A foreign key may accept multiple null values.
Primary key is a clustered index and data in the DBMS table are physically organized in the sequence of the clustered index.	A foreign key cannot automatically create an index, clustered or non-clustered. However, you can manually create an index on the foreign key.
You can have the single Primary key in a table.	You can have multiple foreign keys in a table.

What are Integrity Constraints in DBMS?

- In Database Management Systems, integrity constraints are pre-defined set of rules that are applied on the table fields(columns) or relations to ensure that the overall validity, integrity, and consistency of the data present in the database table is maintained.

Types of Integrity Constraints



There are four types of integrity constraints in DBMS:

- 1.Domain Constraint
- 2.Entity Constraint
- 3.Referential Integrity Constraint
- 4.Key Constraint

Domain Constraint

- Domain integrity constraint contains a certain set of rules or conditions to restrict the kind of attributes or values a column can hold in the database table. The data type of a domain can be string, integer, character, DateTime, currency, etc.

Roll No	Name	Age	Class
101	Adam	14	6
102	Steve	16	8
103	David	8	4
104	Bruce	18	12
105	Tim	6	A

Example:

Consider a Student's table having Roll No, Name, Age, Class of students.

In the student's table, the value A in the last row last column violates the domain integrity constraint because the Class attribute contains only integer values while A is a character.

Referential Integrity Constraints

- Referential Integrity Constraint ensures that there must always exist a valid relationship between two relational database tables. This valid relationship between the two tables confirms that a foreign key exists in a table. It should always reference a corresponding value or attribute in the other table or be null.
- **Example:**
- Consider an Employee and a Department table where Dept_ID acts as a foreign key between the two tables

Referential Integrity Constraints

Employees Table

ID	Name	Salary	Dept_ID
1101	Jackson	40000	3
1102	Harry	60000	2
1103	Steve	80000	4
1104	Ash	1800000	3
1105	James	36000	1

Department Table

Dept_ID	Dept_Name
1	Sales
2	HR
3	Technical

Dept_ID acts as a foreign key in the Employees table and a primary key in the Department table. Row having *DeptID=4* violates the referential integrity constraint since DeptID 4 is not defined as a primary key column in the Departments table.

Entity Integrity Constraint

- Entity Integrity Constraint is used to ensure that the primary key cannot be null. A primary key is used to identify individual records in a table and if the primary key has a null value, then we can't identify those records. There can be null values anywhere in the table except the primary key column.

ID	Name	Salary
1101	Jackson	40000
1102	Harry	60000
1103	Steve	80000
1104	Ash	1800000
	James	36000

In the above employee's table, we can see that the ID column is the primary key and contains a null value in the last row which violates the entity integrity constrain

Key constraint

- Keys are the set of entities that are used to identify an entity within its entity set uniquely. There could be multiple keys in a single entity set, but out of these multiple keys, only one key will be the primary key. A primary key can only contain unique and not null values in the relational database table.

Consider a student's table

Roll No	Name	Age	Class
101	Adam	14	6
102	Steve	16	8
103	David	8	4
104	Bruce	18	12
102	Tim	6	2

The last row of the student's table violates the key integrity constraint since Roll No 102 is repeated twice in the primary key column. A primary key must be unique and not null therefore duplicate values are not allowed in the Roll No column of the above student's table.

Codd's rules

- E.F Codd was a Computer Scientist who invented the **Relational model** for Database management. Based on relational model, the **Relational database** was created. Codd proposed 13 rules popularly known as **Codd's 12 rules** to test DBMS's concept against his relational model.
- Till now, there is hardly any commercial product that follows all the 13 Codd's rules. Even **Oracle** follows only eight and half(8.5) out of 13.

- **Foundation Rule:** For any system that is advertised as, or claimed to be, a relational data base management system, that system must be able to manage data bases entirely through its relational capabilities.
- Rule 1: Information Rule
 - All information in the relational database is represented in exactly one and only one way—by values in tables.

Codd's rules

- Rule 2: Guaranteed Access Rule
 - Each and every datum (atomic value) is guaranteed to be logically accessible by resorting to a combination of table name, primary key value, and column name.
- Rule 3: Systematic Treatment of NULL Values
 - NULL values (distinct from empty character string or a string of blank characters and distinct from zero or any other number) are supported in the fully relational RDBMS for representing missing information in a systematic way, independent of data type.

Codd's rules

- Rule 4: Dynamic Online Catalog Based on the Relational Model
 - The database description is represented at the logical level in the same way as ordinary data, so authorized users can apply the same relational language to its interrogation as they apply to regular data.
- **Rule 5 : Active Online Catalog:** Structure of database must be stored in an online catalog which can be queried by authorized users.
- **Rule 6 : Comprehensive Data Sub-language Rule:** A database should be accessible by a language supported for definition, manipulation and transaction management operation.

Codd's rules

- **Rule 7: -View Updating Rule:** Different views created for various purposes should be automatically updatable by the system.
- **Rule 8 :- High level insert, update and delete rule:** Relational Model should support insert, delete, update etc. operations at each level of relations. Also, set operations like Union, Intersection and minus should be supported.
- **Rule 9 :- Physical data independence:** Any modification in the physical location of a table should not enforce modification at application level.
- .

- **Rule 10 :- Logical data independence:** Any modification in logical or conceptual schema of a table should not enforce modification at application level. For example, merging of two tables into one should not affect application accessing it which is difficult to achieve.
- **Rule 11 :- Integrity Independence:** Integrity constraints modified at database level should not enforce modification at application level.
- **Rule 12:- Distribution Independence:** Distribution of data over various locations should not be visible to end-users.
- **Rule 13:- Non-Subversion Rule:** Low level access to data should not be able to bypass integrity rule to change data.

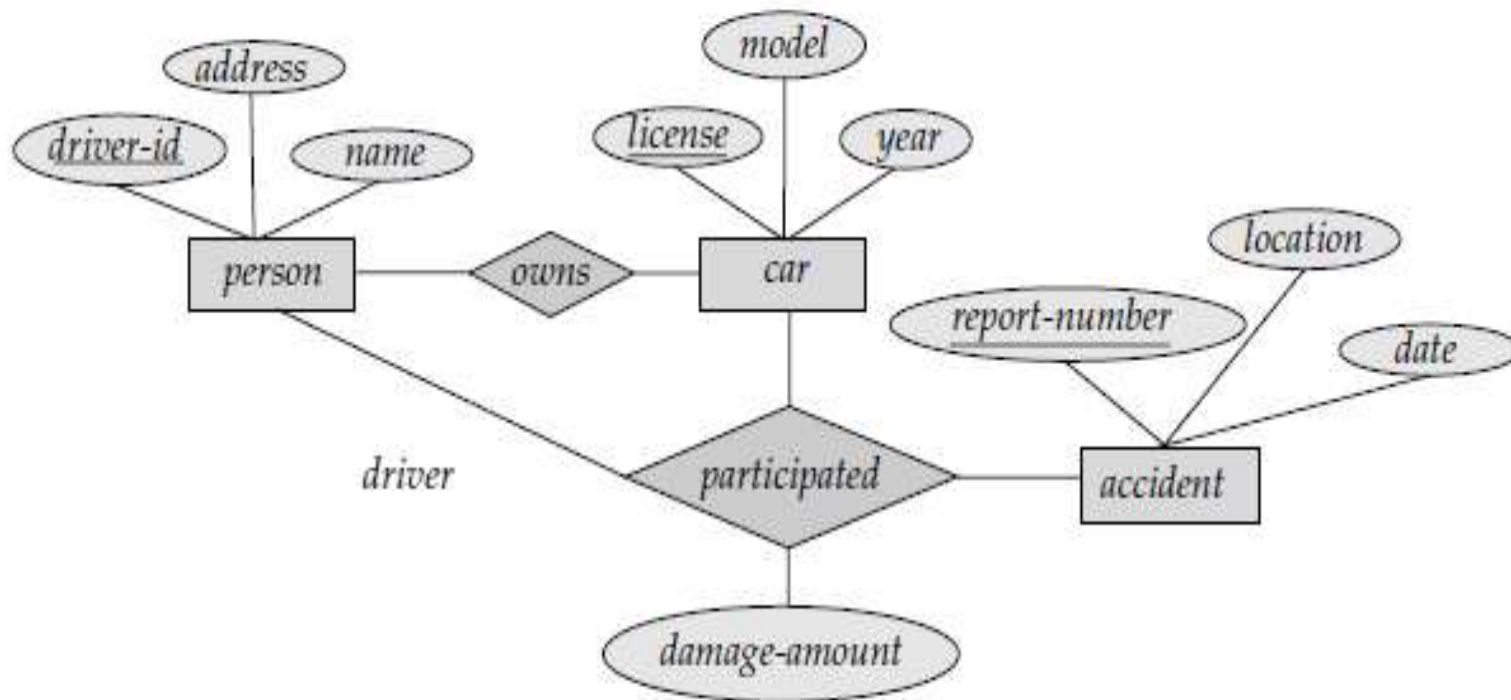


Database Design using E-R, E-R to Relational



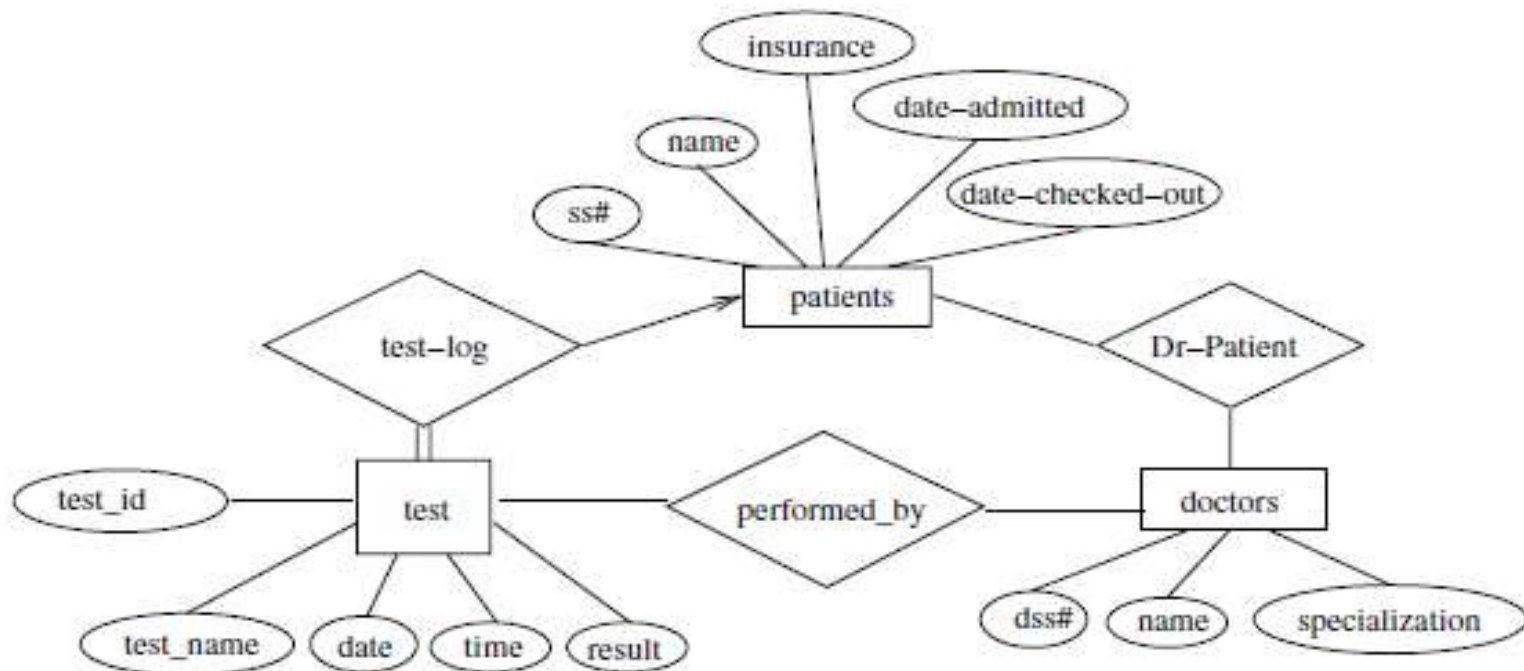
Question

1. Construct an E-R diagram for a car-insurance company whose customers own one or more cars each. Each car has associated with it zero to any number of recorded accidents.



Question

- Construct an E-R diagram for a hospital with a set of patients and a set of medical doctors. Associate with each patient a log of the various tests and examinations conducted..



Question

A university registrar's office maintains data about the following entities:

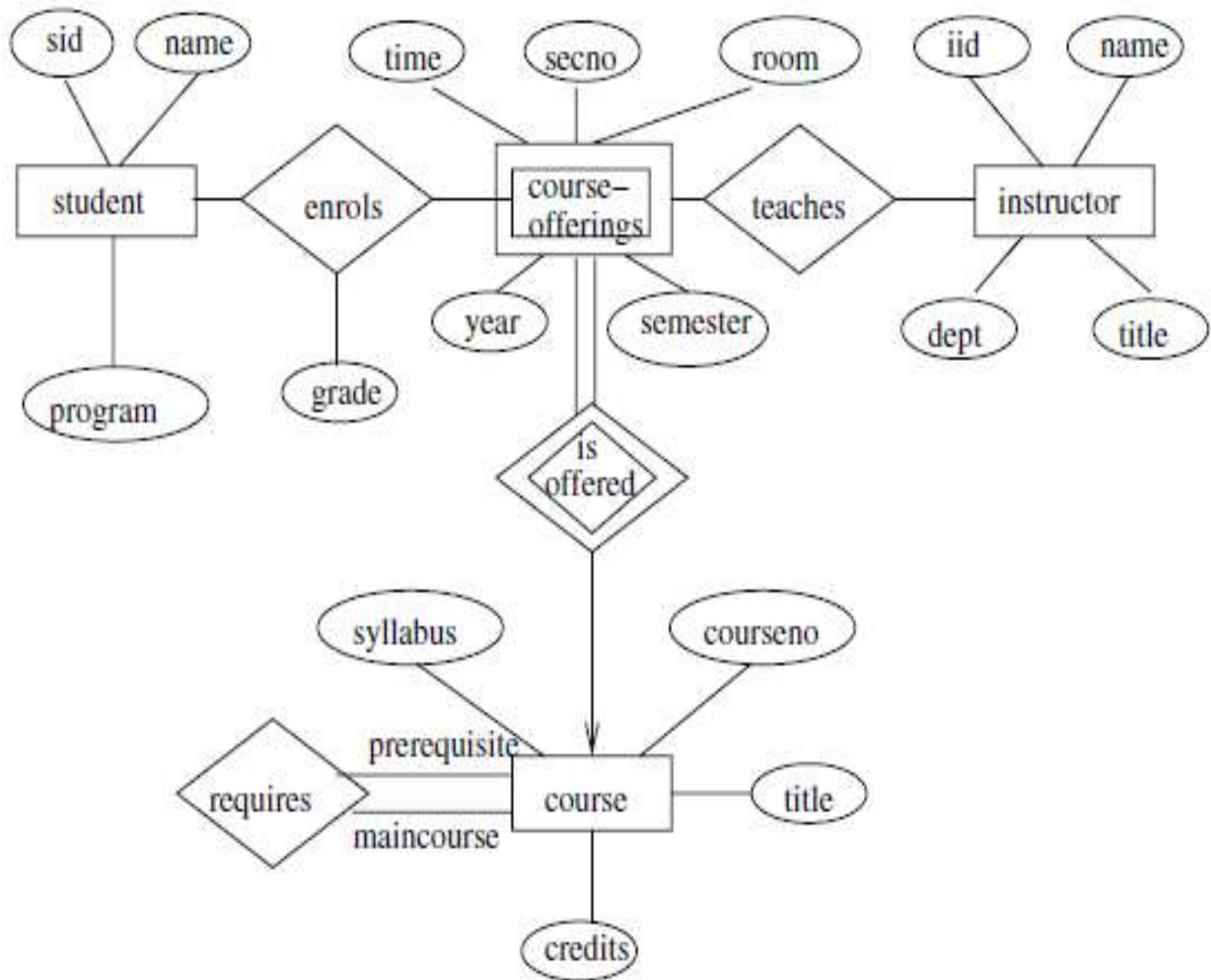
- (a) courses, including number, title, credits, syllabus, and prerequisites;
- (b) Course offerings, including course number, year, semester, section number, instructor(s), timings, and classroom;
- (c) students, including student-id, name, and program;and
- (d) instructors, including identification number, name, department, and title.

Further, the enrollment of students in courses and grades awarded to students in each course they are enrolled for must be appropriately modeled.

Construct an E-R diagram for the registrar's office. Document all assumptions that you make about the mapping constraints.

and *instructor*. The entity set *course-offering* is a weak entity set dependent on *course*. The assumptions made are :

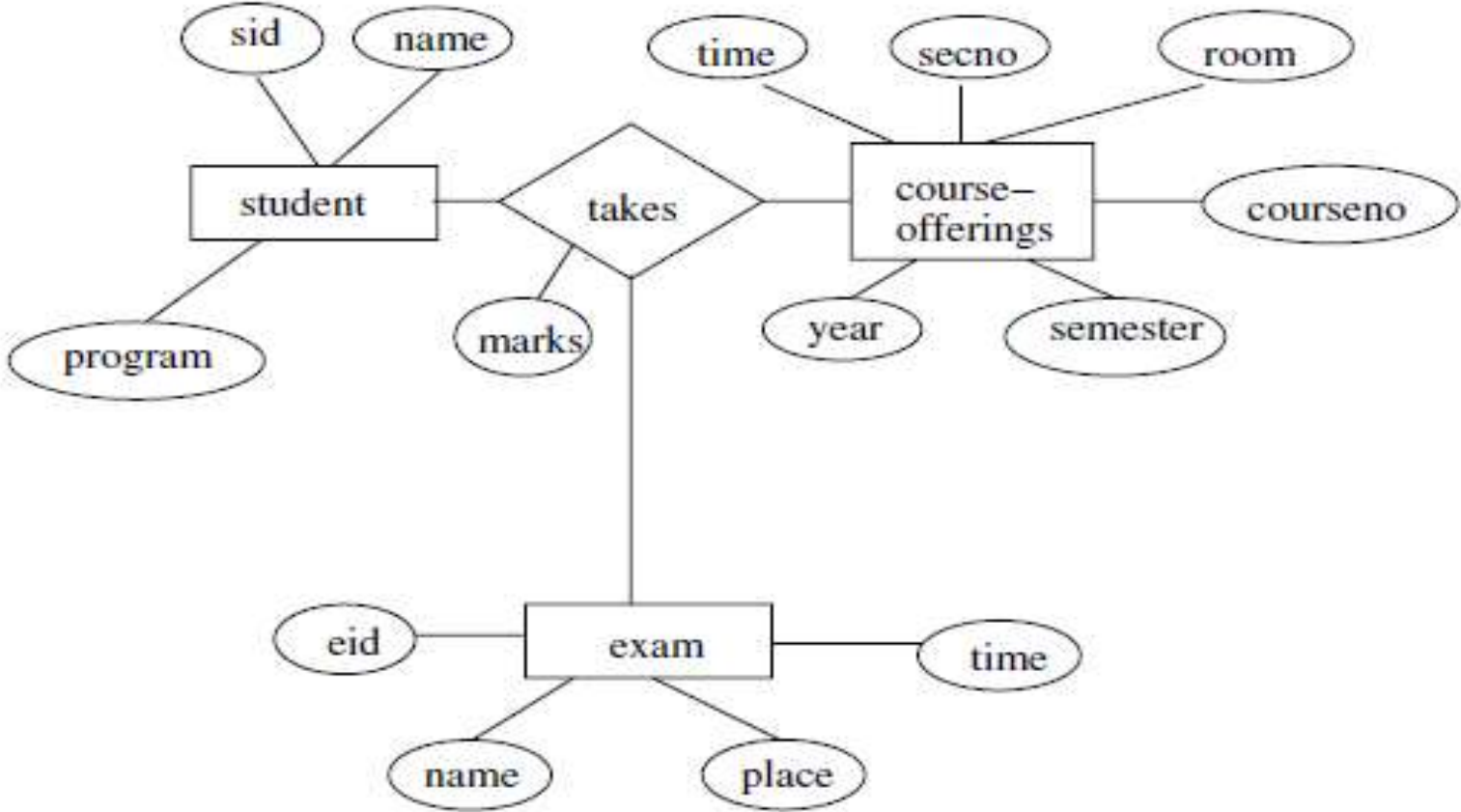
- a.** a class meets only at one particular place and time. This E-R diagram cannot model a class meeting at different places at different times.
- b.** There is no guarantee that the database does not have two classes meeting at the same place and time.

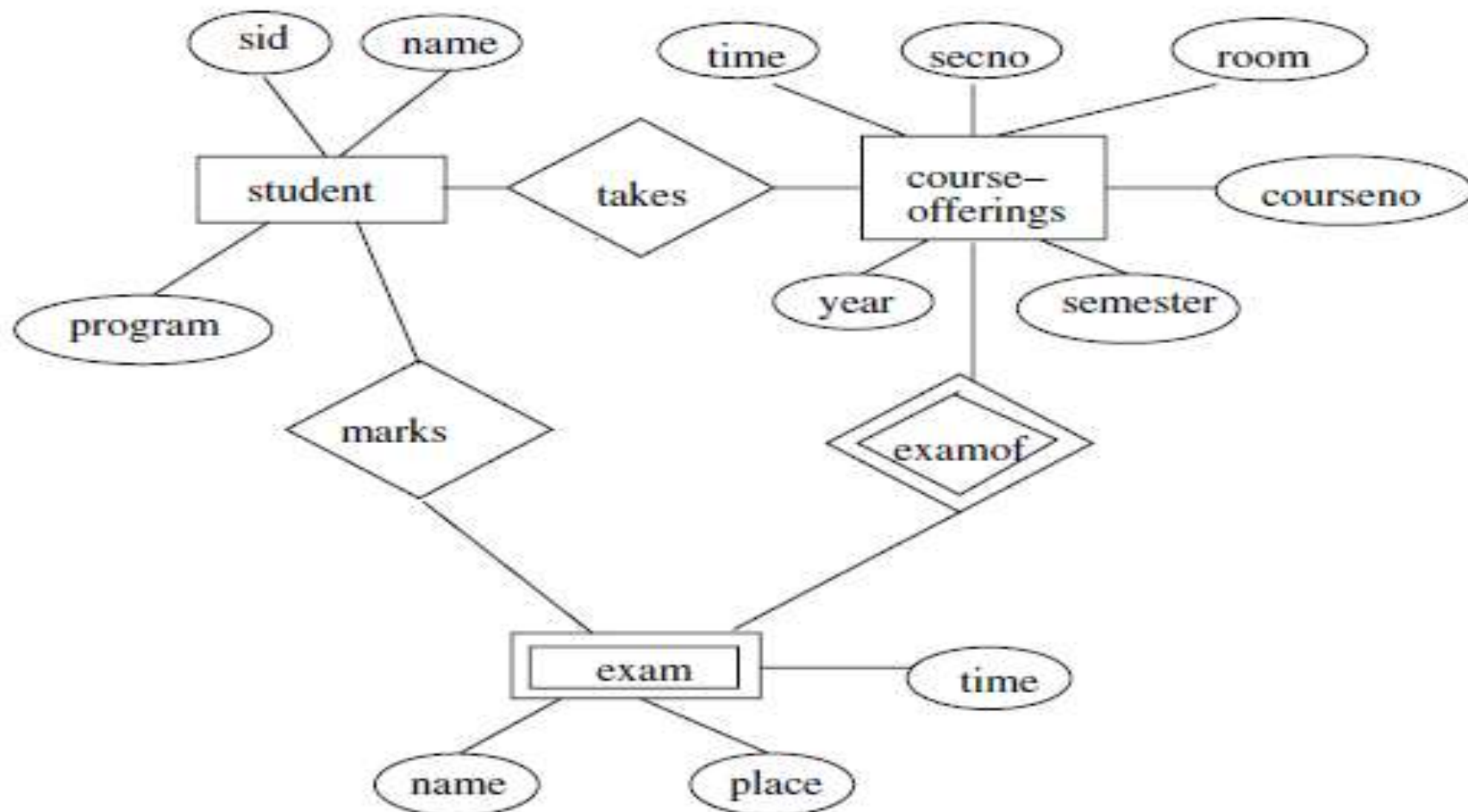


Consider a database used to record the marks that students get in different exams of different course offerings.

- a.** Construct an E-R diagram that models exams as entities, and uses a ternary relationship, for the above database.
- b.** Construct an alternative E-R diagram that uses only a binary relationship between *students* and *course-offerings*. Make sure that only one relationship exists between a particular student and course-offering pair, yet you can represent the marks that a student gets in different exams of a course offering.

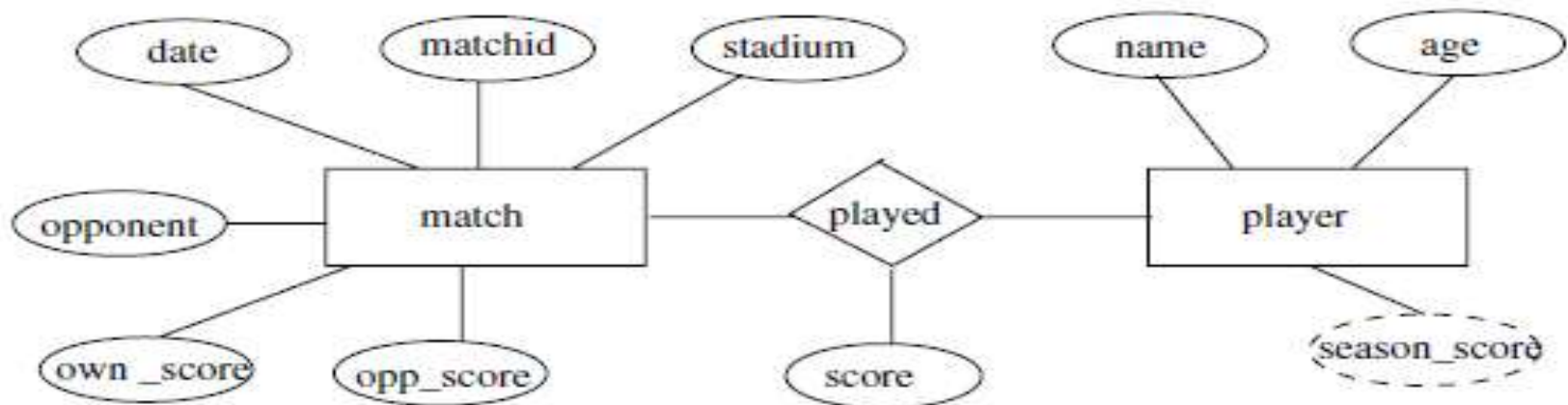
Construct an E-R diagram that models exams as entities, and uses a ternary relationship, for the above database.





Question

Design an E-R diagram for keeping track of the exploits of your favourite sports team. You should store the matches played, the scores in each match, the players in each match and individual player statistics for each match. Summary statistics should be modeled as derived attributes.



Question

1. What is a data model? Draw chart of various data models and write an example of each type of data model.
2. Which data model does not support the many-to-many relationship between entities? Explain that model with advantages and disadvantages.
3. Discuss Network Data Model with its advantages and disadvantages.
4. Describe Relational Model with its advantages. Which is the biggest drawback of this model?
5. Explain object-oriented and object-relational data models with their advantages and disadvantages.

Normalization of Database

- Database Normalization is a technique of organizing the data in the database. Normalization is a systematic approach of decomposing tables to eliminate data redundancy(repetition) and undesirable characteristics like Insertion, Update and Deletion Anomalies. It is a multi-step process that puts data into tabular form, removing duplicated data from the relation tables.
- Normalization is used for mainly two purposes,
 - Eliminating redundant(useless) data.
 - Ensuring data dependencies make sense i.e data is logically stored.

Problems Without Normalization

- If a table is not properly normalized and have data redundancy then it will not only eat up extra memory space but will also make it difficult to handle and update the database, without facing data loss. Insertion, Updation and Deletion Anomalies are very frequent if database is not normalized. To understand these anomalies let us take an example of a **Student** table.

Rollno	name	branch	hod	office_tel
401	Akon	CSE	Mr. X	53337
402	Bkon	CSE	Mr. X	53337
403	Ckon	CSE	Mr. X	53337
404	Dkon	CSE	Mr. X	53337

Problems Without Normalization

Rollno	name	branch	hod	office_tel
401	Akon	CSE	Mr. X	53337
402	Bkon	CSE	Mr. X	53337
403	Ckon	CSE	Mr. X	53337
404	Dkon	CSE	Mr. X	53337

In the table above, we have data of 4 Computer Sci. students. As we can see, data for the fields branch, hod(Head of Department) and office_tel is repeated for the students who are in the same branch in the college, this is Data Redundancy.

Why Do We Need Normalization?

- **Normalization** is a process of organizing the data in database to avoid data redundancy, insertion anomaly, update anomaly & deletion anomaly. Let's discuss about anomalies first then we will discuss normal forms with examples.
- **Anomalies in DBMS**
- There are three types of **anomalies that occur when the database is not normalized**. These are: **Insertion, update and deletion** anomaly.

Why Do We Need Normalization?

- Example: A manufacturing company stores the employee details in a table Employee that has four attributes: Emp_Id for storing employee's id, Emp_Name for storing employee's name, Emp_Address for storing employee's address and Emp_Dept for storing the department details in which the employee works. At some point of time the table looks like

this:

Emp_Id	Emp_Name	Emp_Address	Emp_Dept
101	Rick	Delhi	D001
101	Rick	Delhi	D002
123	Maggie		
166	Glenn		
166	Glenn		

This table is not normalized. We will see the problems that we face when a table in database is not normalized.

Why Do We Need Normalization?

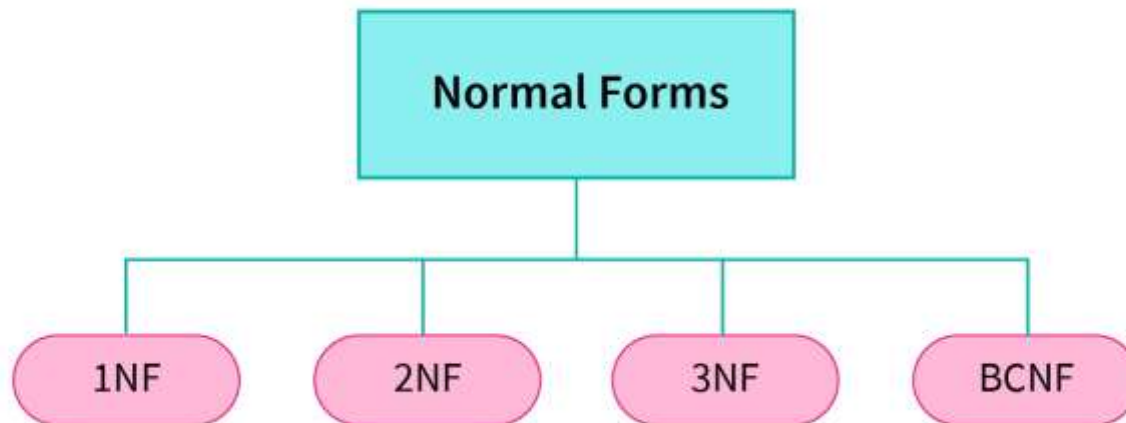
- **Update anomaly:** In the above table we have two rows for employee Rick as he belongs to two departments of the company. If we want to update the address of Rick then we have to update the same in two rows or the data will become inconsistent. If somehow, the correct address gets updated in one department but not in other then as per the database, Rick would be having two different addresses, which is not correct and would lead to inconsistent data.

Why Do We Need Normalization?

- **Insert anomaly:** Suppose a new employee joins the company, who is under training and currently not assigned to any department then we would not be able to insert the data into the table if Emp_Dept field doesn't allow null.
- **Delete anomaly:** Let's say in future, company closes the department D890 then deleting the rows that are having Emp_Dept as D890 would also delete the information of employee Maggie since she is assigned only to this department.
- **To overcome these anomalies we need to normalize the data.**

Normal Forms

- There are four types of normal forms that are usually used in relational databases



First normal form(1NF)

- A relation is said to be in **1NF (first normal form)**, if it doesn't contain any multi-valued attribute. In other words you can say that a relation is in 1NF if each attribute contains only atomic(single) value only.
- As per the rule of first normal form, an attribute (column) of a table cannot hold multiple values. It should hold only atomic values.

First normal form(1NF)

- Example: Let's say a company wants to store the names and contact details of its employees. It creates a table in the database that looks like this:

Emp_Id	Emp_Name	Emp_Address	Emp_Mobile
101	Herschel	New Delhi	8912312390
102	Jon	Kanpur	8812121212 , 9900012222
103	Ron	Chennai	7778881212
104	Lester	Bangalore	9990000123, 8123450987

Two employees (Jon & Lester) have two mobile numbers that caused the Emp_Mobile field to have multiple values for these two employees.

This table is **not in 1NF** as the rule says “each attribute of a table must have atomic (single) values”, the Emp_Mobile values for employees Jon & Lester violates that rule.

First normal form(1NF)

- To make the table complies with 1NF we need to create separate rows for the each mobile number in such a way so that none of the attributes contains multiple values.

Emp_Id	Emp_Name	Emp_Address	Emp_Mobile
101	Herschel	New Delhi	8912312390
102	Jon	Kanpur	8812121212
102	Jon	Kanpur	9900012222
103	Ron	Chennai	7778881212
104	Lester	Bangalore	9990000123
104	Lester	Bangalore	8123450987

Second normal form (2NF)

- Before we learn about the second normal form, we need to understand the following –
- **Prime attribute** – An attribute, which is a part of the candidate-key, is known as a prime attribute.
- **Non-prime attribute** – An attribute, which is not a part of the prime-key, is said to be a non-prime attribute.
- If we follow second normal form, then every non-prime attribute should be fully functionally dependent on prime key attribute. That is, if $X \rightarrow A$ holds, then there should not be any proper subset Y of X , for which $Y \rightarrow A$ also holds true.

Student_Project

Stu_ID

Proj_ID

Stu_Name

Proj_Name

- Student_Project relation that the prime key attributes are Stu_ID and Proj_ID. According to the rule, non-key attributes, i.e. Stu_Name and Proj_Name must be dependent upon both and not on any of the prime key attribute individually. But we find that Stu_Name can be identified by Stu_ID and Proj_Name can be identified by Proj_ID independently. This is called **partial dependency**, which is not allowed in Second Normal Form.

Second Normal Form

- We broke the relation in two as depicted in the above picture.
So there exists no partial dependency.

Student

Stu_ID	Stu_Name	Proj_ID
--------	----------	---------

Project

Proj_ID	Proj_Name
---------	-----------

Second normal form (2NF)

- The normalization of 1NF relations to 2NF involves the elimination of partial dependencies. A partial dependency exists when any non-prime attributes, i.e., an attribute not a part of the candidate key, is not fully functionally dependent on one of the candidate keys.
- For a relational table to be in second normal form, it must satisfy the following rules:
 - The table must be in first normal form.
 - It must not contain any partial dependency, i.e., all non-prime attributes are fully functionally dependent on the primary key.

Second normal form (2NF)

- If a partial dependency exists, we can divide the table to remove the partially dependent attributes and move them to some other table where they fit in well.
- Let us take an example of the following <EmployeeProjectDetail> table to understand what is partial dependency and how to normalize the table to the second normal form:

Employee Code	Project ID	Employee Name	Project Name
101	P03	John	Project103
101	P01	John	Project101
102	P04	Ryan	Project104
103	P02	Stephanie	Project102

Second normal form (2NF)

To remove partial dependencies from this table and normalize it into second normal form, we can decompose the <EmployeeProjectDetail> table into the following three tables:

<EmployeeDetail>

Employee Code	Employee Name
101	John
101	John
102	Ryan
103	Stephanie

<ProjectDetail>

Employee Code	Project ID
101	P03
101	P01
102	P04
103	P02

Project ID	Project Name
P03	Project103
P01	Project101
P04	Project104
P02	Project102

What is Transitive Dependency in DBMS?

- Consider a relation $R(ABC)$, where A , B and C are the attributes of the relation R . A *Transitive dependency* exists when you have the following functional dependency pattern, $A \rightarrow B$ and $B \rightarrow C$; therefore, $A \rightarrow C$.
- In other words, a dependent is indirectly dependent on determinant in Transitive functional dependency.

What is Transitive Dependency in DBMS?

- Let's explore transitive dependency with an example,

- Authors Table**

Author_ID	Author	Book	Author_Nationality
A1	Andrew	The Cursed Villa	United States
A1	Andrew	Children's Mind	United States
A2	Roman	Infinity Tale	Canada

In the Authors Table above:

Book → Author : Here, the *Book* attribute determines the *Author* attribute. If you know the book's name, you can learn the author's name.

Author → Author_Nationality : If we know the *Author* name then we can determine *Author_Nationality* as well.

Book → Author_Nationality : If we know the book name, we can determine the *AuthorsNationality* via the *Author* column.

What is Transitive Dependency in DBMS?

- If you take a closer look into the functional dependencies discussed above, they are indeed forming a pattern
 - $A \rightarrow B$ and $B \rightarrow C$; therefore, $A \rightarrow C$.
 - $A \rightarrow \text{Book}$, $B \rightarrow \text{Author}$ and $C \rightarrow \text{Author_Nationality}$
- To ensure Third Normal Form(3NF) in Authors table, this transitive dependency must be removed and this process of removing transitive dependency is called **Normalization**.

Functional Dependency

- A functional dependency $A \rightarrow B$ in a relation holds if two tuples having same value of attribute A also have same value for attribute B.
- For Example, in relation STUDENT shown in table 1,
Functional Dependencies

STUD_NO- $\rightarrow$ STUD_NAME, STUD_NO- $\rightarrow$ STUD_PHONE hold
but
STUD_NAME- $\rightarrow$ STUD_STATE do not hold

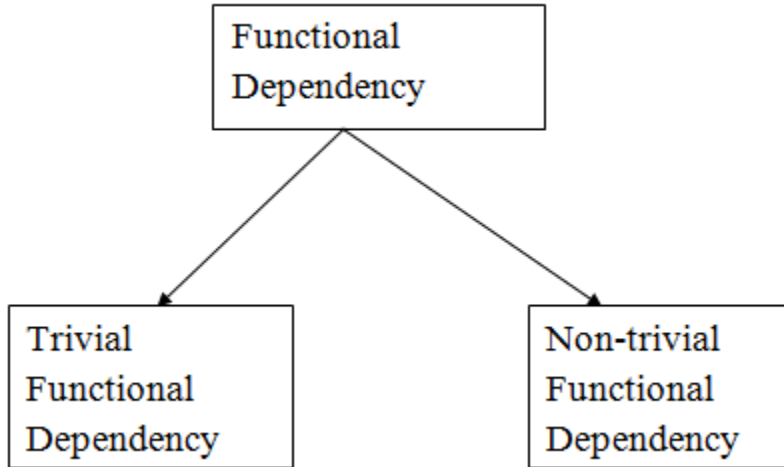
Functional Dependency

STUDENT

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNT RY	STUD_AG E
1	RAM	9716271721	Haryana	India	20
2	RAM	9898291281	Punjab	India	19
3	SUJIT	7898291981	Rajasthan	India	18
4	SURESH		Punjab	India	21

Table 1

Types of Functional dependency



- Trivial functional dependency
- $A \rightarrow B$ has trivial functional dependency if B is a subset of A .
- The following dependencies are also trivial like: $A \rightarrow A$, $B \rightarrow B$

Example:

Consider a table with two columns `Employee_Id` and `Employee_Name`.

$\{\text{Employee_id}, \text{Employee_Name}\} \rightarrow \text{Employee_Id}$ is a trivial functional dependency as `Employee_Id` is a subset of $\{\text{Employee_Id}, \text{Employee_Name}\}$.

Also, $\text{Employee_Id} \rightarrow \text{Employee_Id}$ and $\text{Employee_Name} \rightarrow \text{Employee_Name}$ are trivial dependencies

Types of Functional dependency

- **Non-trivial functional dependency**
- $A \rightarrow B$ has a non-trivial functional dependency if B is not a subset of A .
- When A intersection B is NULL, then $A \rightarrow B$ is called as complete non-trivial.

Example:

ID $\rightarrow$ Name,
Name $\rightarrow$ DOB

Third Normal Form (3NF)

- A relation is in 3NF when it is in 2NF and there is no transitive dependency or a relation is in 3NF, when it is in 2NF and all non-key attributes directly depend on candidate key.
- **Example:-** Consider a relation student (rollno, game, feestructure)

Rollno	Game	Feestructure
1	Basketball	500
2	Basketball	500
3	Basketball	500
4	Cricket	600
5	Cricket	600
6	Cricket	600
7	Tennis	400

Third Normal Form (3NF)

- A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

Third Normal Form (3NF)

- A relation is in third normal form if it holds atleast one of the following conditions for every non-trivial function dependency $X \rightarrow Y$.
 - X is a super key.
 - Y is a prime attribute, i.e., each element of Y is part of some candidate key.
 - If a transitive dependency exists, we can divide the table to remove the transitively dependent attributes and place them to a new table along with a copy of the determinant.

Third Normal Form (3NF)

- <EmployeeDetail> table to understand what is transitive dependency and how to normalize the table to the third normal form:

- **<EmployeeDetail>**

Employee Code	Employee Name	Employee Zipcode	Employee City
101	John	110033	Model Town
101	John	110044	Badarpur
102	Ryan	110028	Naraina
103	Stephanie	110064	Hari Nagar

The above table is not in 3NF because it has

Employee Code -> Employee City transitive dependency because:

Employee Code -> Employee Zipcode

Employee Zipcode -> Employee City

Third Normal Form (3NF)

- <EmployeeDetail> table to understand what is transitive dependency and how to normalize the table to the third normal form:

- <EmployeeDetail>

Employee Code	Employee Name	Employee Zipcode	Employee City
101	John	110033	Model Town
101	John	110044	Badarpur
102	Ryan	110028	Naraina
103	Stephanie	110064	Hari Nagar

The above table is not in 3NF because it has

Employee Code -> Employee City transitive dependency because:

Employee Code -> Employee Zipcode

Employee Zipcode -> Employee City

Also, Employee Zipcode is not a super key and Employee City is not a prime attribute.

Third Normal Form (3NF)

- To remove transitive dependency from this table and normalize it into the third normal form, we can decompose the <EmployeeDetail> table into the following two tables:

- <EmployeeDetail>**

Employee Code	Employee Name	Employee Zipcode
101	John	110033
101	John	110044
102	Ryan	110028
103	Stephanie	110064

<EmployeeLocation>

Employee Zipcode	Employee City
110033	Model Town
110044	Badarpur
110028	Naraina
110064	Hari Nagar

Thus, we've converted the <EmployeeDetail> table into 3NF by decomposing it into <EmployeeDetail> and <EmployeeLocation> tables as they are in 2NF and they don't have any transitive dependency.

Third Normal Form (3NF)

- To remove transitive dependency from this table and normalize it into the third normal form, we can decompose the <EmployeeDetail> table into the following two tables:

- <EmployeeDetail>**

Employee Code	Employee Name	Employee Zipcode
101	John	110033
101	John	110044
102	Ryan	110028
103	Stephanie	110064

<EmployeeLocation>

Employee Zipcode	Employee City
110033	Model Town
110044	Badarpur
110028	Naraina
110064	Hari Nagar

Thus, we've converted the <EmployeeDetail> table into 3NF by decomposing it into <EmployeeDetail> and <EmployeeLocation> tables as they are in 2NF and they don't have any transitive dependency.

Third Normal Form (3NF)

- A relation is in 3NF when it is in 2NF and there is no transitive dependency or a relation is in 3NF, when it is in 2NF and all non-key attributes directly depend on candidate key.
- **Example:-** Consider a relation student (rollno, game, feestructure)

$F - \{ \text{rollno} \rightarrow \text{game}, \text{rollno} \rightarrow \text{feestructure}, \text{game} \rightarrow \text{fee} \}$
 $\text{Rollno}^+ = \{ \text{rollno}, \text{game}, \text{feestructure} \}$
 $\Rightarrow \text{rollno}$ is primary key

The above student table is in 1NF because there are no multivalued attributes.

Rollno	Game	Feestructure
1	Basketball	500
2	Basketball	500
3	Basketball	500
4	Cricket	600
5	Cricket	600
6	Cricket	600
7	Tennis	400

Third Normal Form (3NF)

- Student table is also in 2NF because all non-key attributes are fully functional dependent on the primary key (rollno). But the table is not in 3NF because there is transitive dependency i.e. game-> feestructure.
- Feestructure has transitive/indirect dependency on rollno via game.

Boyce and Codd Normal Form (BCNF)

- BCNF is the advance version of 3NF. It is stricter than 3NF.
- A table is in BCNF if every functional dependency $X \rightarrow Y$, X is the super key of the table.
- For BCNF, the table should be in 3NF, and for every FD, LHS is super key.
- **Example:** Let's assume there is a company where employees work in more than one department.

Boyce and Codd Normal Form (BCNF)

EMPLOYEE table:

EMP_ID	EMP_COUNTRY	EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
264	India	Designing	D394	283
264	India	Testing	D394	300
364	UK	Stores	D283	232
364	UK	Developing	D283	549

In the above table Functional dependencies are as follows:

$EMP_ID \rightarrow EMP_COUNTRY$

$EMP_DEPT \rightarrow \{DEPT_TYPE, EMP_DEPT_NO\}$

Candidate key: {EMP-ID, EMP-DEPT}

The table is not in BCNF because neither EMP_DEPT nor EMP_ID alone are keys.

To convert the given table into BCNF, we decompose it into three tables:

Boyce and Codd Normal Form (BCNF)

EMP_COUNTRY table:

EMP_ID	EMP_COUNTRY
264	India
264	India

EMP_DEPT_MAPPING table:

EMP_ID	EMP_DEPT
D394	283
D394	300
D283	232
D283	549

EMP_DEPT table:

EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
Designing	D394	283
Testing	D394	300
Stores	D283	232
Developing	D283	549

Functional dependencies:

$EMP_ID \rightarrow EMP_COUNTRY$

$EMP_DEPT \rightarrow \{DEPT_TYPE, EMP_DEPT_NO\}$

Candidate keys:

For the first table: EMP_ID

For the second table: EMP_DEPT

For the third table: {EMP_ID, EMP_DEPT}

Now, this is in BCNF because left side part of both the functional dependencies is a key.



Question

6. Which are the three level of database architecture? Explain each in detail with diagram.
7. Write a brief note on database languages.
8. Differentiate between data independence and structural independence

Question

6. Explain the distinctions among the terms primary key, candidate key, and superkey

ANS :- Answer: A superkey is a set of one or more attributes that, taken collectively, allows us to identify uniquely an entity in the entity set. A superkey may contain extraneous attributes. If K is a superkey, then so is any superset of K . A superkey for which no proper subset is also a superkey is called a candidate key. It is possible that several distinct sets of attributes could serve as candidate keys. The primary key is one of the candidate keys that is chosen by the database designer as the principal means of identifying entities within an entity set.